

Investigations into Schema Linking in Text-to-SQL

— *A Comparative Evaluation of Six Schema Linking Methods on the Spider Dataset*

Submitted by:

Mufaddal Ragib

Matriculation No: 102163

to the Department VI – Informatik & Medien
of Berliner Hochschule für Technik

A thesis submitted in partial fulfillment
of the requirements for the degree of
Master of Science (M.Sc.)
in study program
Data Science

Date of submission: **1st September 2026**



Supervisors

1. Prof. Dr. Patrick Erdelt Berliner Hochschule für Technik
2. Prof. Dr. Steffen Wagner Berliner Hochschule für Technik

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Prof. Dr. Patrick Erdelt, whose guidance shaped this thesis from its first vague idea to its final form. His questions consistently pushed me to think more carefully, and his feedback at each stage kept the work focused and rigorous.

I am particularly grateful for the human-annotated dataset he provided to me, which formed the empirical foundation of this study and without which the evaluation presented here would not have been possible. I also thank him for pointing me to several key papers early on; they gave me a footing in the literature far more quickly than I would have found on my own.

Beyond the practical support, I am thankful for his patience and encouragement throughout the process.

Declaration of Authorship

I hereby declare that I have written this master's thesis independently and without any unauthorized external assistance. All sources and materials used have been fully and correctly acknowledged, and any passages quoted or paraphrased from other works have been clearly identified and cited.

1st September 2026

Date

Mufaddal Ragib

Signature

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Scope and Contributions	2
1.4	Thesis Structure	2
2	Background and Related Work	3
2.1	Text-to-SQL: Overview and Pipeline	3
2.2	Schema Linking: Definitions and Role	3
2.3	Existing Schema Linking Approaches	4
2.3.1	Lexical / Rule-Based Methods	4
2.3.2	Embedding-Based Methods	4
2.3.3	LLM-Based Methods (Forward, Backward, Bidirectional)	5
2.3.4	Graph-Based Methods	5
2.4	Evaluation Practices for Schema Linking in Prior Work	6
2.5	Gaps in Existing Literature	6
3	Dataset and Gold-Standard Construction	8
3.1	The Spider Dataset	8
3.2	Gold-Link Annotation Approaches	11
3.2.1	Tier 1 (Mentioned): Human Annotations (Taniguchi et al., 2021)	11
3.2.2	Tier 2 (All-SQL-Used): sqlglot-Extracted Links	13
3.3	What Tier 2 Contains	15
3.4	Dataset Statistics and Splits	18
4	Methodology	20
4.1	Problem Formulation	20
4.2	Schema Linking Methods Under Comparison	20
4.2.1	Method A - Lexical Matching	21
4.2.2	Method B - Embedding-Based Matching	23
4.2.3	Method C - LLM-Forward	25
4.2.4	Method D - LLM-Backward	26
4.2.5	Method E - LLM-Bidirectional	27
4.2.6	Method G - Graph-Based	28
4.3	Shared Infrastructure	32
4.3.1	Linker Protocol (base.py)	32
4.3.2	Shared SQL-Parsing Utilities	33
4.3.3	LLM Client: Retries, Cost Tracking, and Caching	33
4.4	Threshold Tuning, Few-Shot Selection, and Prompt Iteration	33
4.4.1	Method A and B: Threshold and Grid Tuning	33
4.4.2	Few-Shot Example Selection	35

4.5	Evaluation Metrics	37
4.5.1	Precision, Recall, F1	37
4.5.2	Strict Retrieval Rate (SRR)	38
4.5.3	F-beta ($\beta = 6$)	38
4.5.4	Hallucination Rate (HR)	39
4.6	Statistical Testing (McNemar's Test)	39
5	Results	41
5.1	Overall Performance Comparison Across Methods	41
5.2	Per-Method Analysis	41
5.3	Tier 1 vs. Tier 2 Gold-Standard Comparison	47
5.4	Statistical Significance (Paired McNemar's Tests)	48
5.5	Error Analysis	51
6	Discussion	54
6.1	String Matching versus Embeddings	54
6.2	Do LLM Methods Win, and at What Cost?	54
6.3	Does the Graph Method Close the Gap?	55
6.4	Dominant Error Categories	56
7	Conclusion and Future Work	57
7.1	Summary of Findings	57
7.2	Contributions	57
7.3	Future Work	58
	References	60
A	Gold-Link JSON Schema	62
B	Prompt Templates for LLM-Based Methods	63
B.1	Method C - forward_v1	63
B.2	Method G - graph_endpoint_v1	63
B.3	Method D - backward_v1	64
C	Further Worked Schema-Linking Examples	65
D	Implementation Reference	67

List of Figures

3.1	Database size across the 166 schemas in Spider’s tables.json	9
3.2	The 1,034 Spider development questions	9
3.3	Share of the 1,034 development queries carrying each SQL construct	10
3.4	Clause co-occurrence across the 1,034 development queries	11
3.5	Where each tier’s links come from, on development question 22...	12
3.6	How one gold query becomes a set of Tier-2 links, worked on development... .	13
3.7	Gold columns per development question	15
3.8	Where the two gold tiers agree, and where they do not, pooled over the 1,034... .	16
3.9	The same two distributions	16
3.10	Gold columns per development question by Spider hardness	17
3.11	The reverse asymmetry, on development question 6	18
4.1	Method A on development question 39	22
4.2	Method B’s two conditions on one development question	24
4.3	What the bidirectional union buys, pooled over the 1,034 development questions .	28
4.4	Method G’s two stages	29
4.5	The greedy Steiner-tree approximation on dog_kennels	30
4.6	Macro F1 against fuzzy_threshold, swept from 40 to 95 on the seeded...	35
4.7	Method B’s 400-point tuning grid	36
4.8	The five metrics on one prediction, Method B on development question 4	37
4.9	How F6 responds, compared with F1	39
5.1	Every row of Table 5.1 at the column level, as one point per method and tier . . .	42
5.2	Mean columns predicted per development question, by method, against both gold... .	42
5.3	The whole of Method D’s hallucination rate, on development question 26	45
5.4	Column-level strict recall by Spider hardness, one line per method, Tier 1 in... .	46
5.5	How far F1 and strict recall disagree, by method and hardness, on Tier-2... . . .	47
5.6	Column-level strict recall under each tier, one line per method	48
5.7	All 32 paired McNemar tests	49
5.8	The distribution of classified recall failures	51
C.1	Development question 12 (concert_singer): the straightforward case, in which... .	65
C.2	Development question 898 (network_1): a set operation, with each branch of... .	66
C.3	Development question 77 (pets_1): a three-table join whose bridge table the... .	66

Chapter 1

Introduction

1.1 Motivation

Relational databases store the majority of the world’s structured information, but querying them requires fluency in SQL, a barrier that excludes the vast majority of potential users. Text-to-SQL systems address this barrier by translating natural language questions directly into executable SQL queries, and the task has attracted sustained attention from both the natural language processing and database communities (Qin et al., 2022; Liu et al., 2024). As the large language models (LLMs) powering modern Text-to-SQL systems have grown more capable, a persistent bottleneck has moved into focus: real-world database schemas can span hundreds of tables and thousands of columns, far more than can be usefully placed in a single prompt, and far more than any one question actually needs. Schema linking - identifying which tables and columns in a schema are relevant to a given question - has therefore become a critical sub-task within the Text-to-SQL pipeline.

The arrival of long-context LLMs has complicated, rather than resolved, this picture. Maamari et al. (2024) argue that for sufficiently capable models with schemas that fit comfortably in context, explicit schema linking may cost more accuracy than it saves, since imperfect filtering risks excluding a column the model actually needed. Yet even that work concedes the opposite holds for smaller or weaker models, and for schemas that do not fit in context at all - a scenario that is the norm rather than the exception in enterprise settings, where databases with thousands of columns are common (Wang et al., 2025b). Schema linking, in other words, has not become obsolete; its value has become conditional - on model capability, schema scale, and deployment cost - and that conditionality is precisely what makes a controlled comparison of methods useful now.

Despite this proliferation of methods, direct comparison between them remains difficult. Most papers introduce one new method and compare it against a small, non-uniform set of baselines, under whatever gold-standard definition and metric set that paper’s authors happened to adopt. Glass et al. (2025) note explicitly that no standard metric for schema linking evaluation exists: some works report ROC-AUC, others report precision and recall, and even the ground truth those metrics are computed against differs from paper to paper. Katsogiannis-Meimarakis et al. (2026) raise a related concern specifically for LLM-based linkers, showing that basic design choices - how many few-shot demonstrations to use, how many outputs to sample - shift the precision-recall trade-off substantially, in ways that are easy to miss without systematic ablation. The result is a literature in which it is genuinely difficult to say, with any confidence, how a cheap lexical baseline actually compares to an LLM-based linker, or whether the added cost of an LLM call is justified by the accuracy it buys.

1.2 Problem Statement

Despite broad agreement that schema linking materially affects Text-to-SQL performance, no existing study evaluates the major families of schema linking methods - lexical, embedding-based, and the several distinct ways an LLM can be used (reasoning forward from the question, backward from a draft query, or bidirectionally), alongside graph-based approaches - on a common dataset, against a common gold standard, under a common set of metrics. This absence of a controlled, like-for-like comparison leaves basic practical questions unresolved: how much accuracy an LLM-based linker actually buys over a cheap lexical or embedding baseline, whether that gap is large enough to justify the added latency and API cost, and whether a graph-based method offers a viable lower-cost alternative to LLM-based reasoning at all. This thesis addresses these questions by isolating schema linking from the rest of the Text-to-SQL pipeline and evaluating six representative methods under one shared protocol, so that differences in measured performance and in error pattern can be attributed to the linking method itself rather than to confounding differences in SQL generation, execution, or fine-tuning procedure.

1.3 Scope and Contributions

This thesis is deliberately scoped to the schema linking sub-task in isolation. It does not address full Text-to-SQL query generation, does not fine-tune any language or embedding model, and does not evaluate execution accuracy of downstream SQL. This narrower scope is a deliberate methodological choice: by holding the rest of the Text-to-SQL pipeline fixed and out of scope, differences observed between methods can be attributed to schema linking quality specifically, rather than to interactions with a particular SQL generator, decoding strategy, or fine-tuning regime. All experiments are conducted on the Spider dataset (Yu et al., 2018), the most widely used cross-domain benchmark for this line of research.

1.4 Thesis Structure

The remainder of this thesis is organized as follows.

Chapter 2 reviews related work on Text-to-SQL and schema linking in greater depth, situating the six method families evaluated here within the broader literature and identifying the specific evaluation gaps this thesis addresses.

Chapter 3 describes the Spider dataset and the construction of the hybrid two-tier gold standard, including the provenance and limitations of each tier.

Chapter 4 presents methodology: a formal problem formulation, each of the six schema linking methods together with its exact implementation and tuning, the shared linker protocol and SQL-parsing infrastructure they have in common, the evaluation metrics and statistical test used throughout, and the cost-management and reproducibility framework governing every LLM call.

Chapter 5 reports results across all methods, metrics, and gold-standard tiers, together with paired significance testing and a full error.

Chapter 6 discusses what those results mean, organized around this thesis’s research questions, and considers the study’s limitations and threats to validity.

Chapter 7 concludes with a summary of findings, contributions, and directions for future work.¹

¹All code, configuration, prompts, predictions, and result tables underlying this thesis are publicly available at <https://github.com/Mufaddalsr/schema-linking-text-to-sql>.

Chapter 2

Background and Related Work

2.1 Text-to-SQL: Overview and Pipeline

Given a natural language question Q and a database schema $S = \{T, C, R\}$, where T is a set of tables, C a set of columns, and R a set of foreign-key relations, the Text-to-SQL task seeks a function that maps (Q, S) to an executable SQL query Y that correctly answers Q against the database (Dong et al., 2023; Safdarian et al., 2025). Historically, Text-to-SQL research has passed through three broad stages. Early rule-based and grammar-based systems from the database community achieved reasonable accuracy only within narrow, single-domain settings, at the cost of extensive hand-engineering (Qin et al., 2022). The introduction of large annotated cross-domain benchmarks, most influentially Spider (Yu et al., 2018) - shifted the field toward pre-trained language model (PLM) based encoder-decoder architectures, which learned to jointly encode the question and schema and decode a SQL query, resolving roughly 80% of Spider’s cases while still struggling on its hardest queries (Liu et al., 2024). The most recent stage is dominated by large language models (LLMs), which perform Text-to-SQL largely through prompting rather than task-specific fine-tuning, pushing execution accuracy considerably higher while introducing new practical constraints around prompt length and inference cost (Liu et al., 2024; Dong et al., 2023).

Across all three stages, a common pipeline has emerged: a retrieval or schema-linking stage narrows the full database schema to a relevant subset; a generation stage produces a candidate SQL query conditioned on the question and the retrieved schema; and increasingly in the LLM era, a correction stage regenerates or repairs the query based on execution feedback or self-critique (Maamari et al., 2024). This thesis is concerned exclusively with the first stage.

2.2 Schema Linking: Definitions and Role

Schema linking is the sub-task of identifying which tables and columns of a database schema are relevant to a given natural language question (Lei et al., 2020; Wang et al., 2020). Safdarian et al. (2025) formalize it as a function that maps a question and a schema graph to a connected sub-schema sufficient to express the SQL query answering that question. Two motivations for this sub-task recur throughout the literature. The first is computational and economic: real-world schemas can contain hundreds of tables and thousands of columns, more than fits inside a model’s context window, and every additional schema token included in a prompt has a direct cost in latency and API spend (Glass et al., 2025; Wang et al., 2025b). The second is accuracy: even when a schema does fit in context, including irrelevant tables and columns has been shown to

actively degrade generation quality, since models can mistakenly select or join against irrelevant columns (Dong et al., 2023; Maamari et al., 2024).

2.3 Existing Schema Linking Approaches

Prior work on schema linking falls into four broad families, corresponding to the six methods evaluated in this thesis: lexical/rule-based matching, embedding or ranking-based neural matching, LLM-based reasoning (in forward, backward, and bidirectional variants), and graph-based structural matching.

2.3.1 Lexical / Rule-Based Methods

The earliest schema-linking components relied on surface-level string matching between question tokens and schema names, typically implemented as n-gram overlap or edit-distance heuristics. IRNet (Guo et al., 2019), for instance, augments its encoder with hand-crafted type vectors derived from exactly this kind of string matching. These heuristics are attractive for being training-free and essentially instantaneous, but their limitations are well documented. Lei et al. (2020) benchmark an automatic n-gram matching annotator of this kind against their own manually corrected gold standard and report only moderate agreement - an F1 of 0.775 for column references, 0.801 for tables - with most of the shortfall traced to paraphrases (a column named elevation referred to in the question as “altitude”) and partial mentions that simple matching rules do not generalize to. Taniguchi et al. (2021) reach a similar conclusion from their own independent annotation effort, motivating human annotation specifically because lexical heuristics were judged too noisy to serve as ground truth on their own. This body of evidence is precisely why the present thesis treats lexical matching as a baseline lower bound (Method A) rather than a candidate for the gold standard itself, and why its programmatically extracted Tier 2 gold standard is instead derived by parsing the gold SQL query rather than by string-matching the question.

2.3.2 Embedding-Based Methods

A second family replaces exact string overlap with a learned or pre-trained notion of semantic similarity between question and schema text. Li et al. (2023) train a cross-encoder that jointly scores every table and column against the question and retains only the top-ranked four tables and five columns per table, reporting that this ranking step alone provides most of the benefit of their overall RESDSQL system. Wang et al. (2025a) take a retrieval-first approach at a larger scale, using multi-round semantic retrieval to narrow a pool of candidate databases and schema items before a separate grounding stage identifies the precise tables and columns needed, specifically to handle databases with thousands of fields spread across many separate schemas. Glass et al. (2025) occupy a middle position: their extractive linker reuses the token-level hidden representations of a decoder-only LLM as a similarity signal, rather than training a dedicated embedding model, giving fine-grained control over the precision-recall trade-off at inference time without a generative decoding step. The embedding-based method evaluated in this thesis (Method B) follows the spirit of this family - scoring schema items by embedding similarity to the question rather than by exact lexical overlap - while deliberately using an off-the-shelf sentence embedding model rather than a task-specific fine-tuned cross-encoder, in order to isolate what a lightweight, training-free semantic method buys over lexical matching without confounding the comparison with a fine-tuning budget the other methods do not receive.

2.3.3 LLM-Based Methods (Forward, Backward, Bidirectional)

The most active area of recent schema-linking research uses an LLM directly as the linking mechanism, and the literature splits fairly cleanly along the direction of reasoning involved. *Forward methods* reason from the question toward the schema: Dong et al. (2023) C3 system prompts an LLM in a zero-shot setting to rank tables and columns by relevance to the question and retains the top candidates, reporting AUC scores of 0.9588 for table recall and 0.9833 for column recall on Spider’s development set - evidence that a single well-prompted LLM call can rival specialized rankers without any fine-tuning. *Backward methods* instead reason from an initial, possibly imperfect SQL query back to the schema: Yang et al. (2024) SQL-to-Schema framework prompts an LLM to draft a full SQL query against the complete schema first, then parses that draft to recover the specific tables and columns it references, on the reasoning that a model reasoning about what a query would need to do is a more reliable schema-selection signal than a model reasoning about what a question is asking about in isolation. *Bidirectional methods* combine both signals: Cao et al. (2024) RSL-SQL runs forward and backward linking in parallel and unions the results, and their own ablation shows this union outperforms either direction alone for GPT-4o, though for a weaker DeepSeek-based model, backward linking alone slightly outperforms the bidirectional union, since the added recall from combining directions also introduces noise a weaker model handles less gracefully - evidence that the value of combining directions is itself model-dependent, not a fixed property of the technique.

A further LLM-based direction goes beyond a single forward or backward pass and treats schema exploration as an iterative or agentic process. Wang et al. (2025b) AutoLink dynamically expands a linked schema subset over several LLM-guided steps rather than committing to a single-shot prediction, reporting strict recall above 97% on the BIRD development set specifically because iterative expansion allows the model to recover from an initially incomplete guess. This body of work also speaks directly to two design choices this thesis makes for its own LLM-based methods. First, on sampling: Katsogiannis-Meimarakis et al. (2026) show that increasing the number of sampled LLM outputs and taking their union systematically increases recall at the cost of precision, and Dong et al. (2023) report a comparable pattern when varying the number of self-consistency samples for C3 - both consistent with this thesis’s choice to use a small, fixed self-consistency budget ($k = 3$) for its forward LLM method rather than treating sample count as an unbounded lever for chasing recall. Second, on few-shot demonstrations: Katsogiannis-Meimarakis et al. (2026) find that a single demonstration example meaningfully helps, but that additional demonstrations beyond one help inconsistently across models, cautioning against assuming that more in-context examples reliably improve LLM-based linking.

2.3.4 Graph-Based Methods

The final family exploits a structural property of relational schemas that the other three largely ignore: schemas are graphs, with tables as nodes and foreign-key relations as edges. Two rather different design points populate this family. Wang et al. (2020) RAT-SQL treats the schema graph as an input to a neural relation-aware self-attention mechanism, learning soft, jointly-trained representations of question tokens and schema elements that are biased by, but not strictly limited to, the schema’s actual foreign-key structure; this places RAT-SQL at the boundary between the graph-based and embedding-based families, since its schema linking is ultimately a byproduct of a trained encoder rather than a symbolic graph operation. Safdarian et al. (2025) SchemaGraph-SQL takes the opposite, training-free position: it uses a single LLM call only to identify coarse source and destination tables, then applies classical deterministic shortest-path algorithms over the schema’s foreign-key graph to recover the full connected sub-schema joining them, reporting that this purely algorithmic approach outperforms specialized fine-tuned and prompt-heavy

linkers on BIRD, with the path-finding step itself adding under 15 milliseconds of latency on top of a single lightweight LLM call. RAT-SQL and its more recent graph-neural descendants - including ShadowGNN (Chen et al., 2021), LGE SQL (Cao et al., 2021), and SADGA (Cai et al., 2021) - represent a mature but comparatively heavyweight line of work requiring task-specific training; SchemaGraphSQL’s classical path-finding represents a newer, lighter-weight alternative. The graph-based method evaluated in this thesis (Method G) follows this second, training-free tradition, since it is the one most compatible with this thesis’s scope of comparing methods without a dedicated fine-tuning budget for any single family.

2.4 Evaluation Practices for Schema Linking in Prior Work

A recurring theme across the works reviewed above is the absence of a shared evaluation convention, a point Glass et al. (2025) make explicitly: some schema-linking papers report area under the ROC curve, others report precision and recall, and the definitions of “correct” against which those metrics are computed are not the same from paper to paper. The papers surveyed in this chapter illustrate the point concretely rather than abstractly. Li et al. (2023) and Dong et al. (2023) both report AUC for table and column recall. Lei et al. (2020) and Taniguchi et al. (2021) each report precision, recall, and F1, but against two independently constructed gold standards that need not agree with one another on borderline cases. Cao et al. (2024) introduce Strict Recall Rate, a stricter all-or-nothing variant of recall computed at the level of an entire column combination rather than per-column. Yang et al. (2024) instead report table-recall@4, a metric defined relative to a fixed candidate-set size rather than an open-ended prediction. Glass et al. (2025) argue for recall-weighted F-beta specifically because, in their own end-to-end experiments, the F6 score was well correlated with downstream SQL generation accuracy, while precision alone was not similarly predictive, motivating their recommendation that schema-linking evaluation should weight recall heavily rather than treating precision and recall as equally important.

This inconsistency is not merely a matter of academic bookkeeping. Because each paper’s reported numbers are computed against a metric and a gold standard chosen by that paper’s own authors, cross-paper comparison of the kind this thesis undertakes, asking whether an LLM-based method meaningfully outperforms a lexical or embedding-based one cannot be done reliably using published numbers alone; the methods have to be re-run and re-scored under one shared protocol. This is precisely the gap this thesis’s five-metric evaluation framework (precision, recall, F1, SRR, and F6) is designed to close: rather than picking one of these conventions as authoritative, it computes all of them for every method, against both a human-annotated (Tier 1) and a programmatically derived (Tier 2) gold standard, so that a method’s ranking can be checked for robustness across metric and gold-standard choice rather than being an artifact of whichever single metric happened to be reported.

2.5 Gaps in Existing Literature

Three gaps emerge from this review, each of which this thesis is designed to address directly.

- First, no work surveyed here evaluates lexical, embedding-based, LLM-based, and graph-based schema linking methods against one another under a single protocol; each paper instead introduces one new method and compares it to a small set of baselines drawn from whichever prior papers are most convenient, typically from only one or two of the four families.

- Second, as documented in Section 2.4, the metrics and gold-standard definitions used to support those comparisons are themselves inconsistent across papers, which means that even a careful reader assembling numbers from multiple papers by hand cannot be confident the resulting comparison is fair.
- Third, cost and practicality are rarely reported alongside accuracy: most of the LLM-based papers reviewed here report execution accuracy or schema-linking F-scores, but few report the API cost, latency, or engineering complexity of the method being proposed, making it difficult for a practitioner to weigh a small accuracy gain against a potentially large cost increase.

The “does schema linking still help” debate discussed in Section 2.2 is itself a symptom of this last gap: Maamari et al. (2024) and the large-scale schema-linking papers that motivate the opposite conclusion (Wang et al., 2025b,a) are not actually in direct disagreement so much as they are answering the question under different, unstated assumptions about schema scale and model capability.

Chapter 3

Dataset and Gold-Standard Construction

Every metric reported in this thesis is computed against a gold standard, so how that standard is constructed determines what the results in Chapter 5 can be said to measure.

Section 3.1 describes the Spider dataset as this thesis’s own pipeline loads it, together with the constraint that shapes every choice downstream: gold SQL is released for the training and development splits but withheld for the test split. Section 3.2 sets out the two gold-link tiers - Tier 1, from Taniguchi et al. (2021) human annotations of what a question mentions, and Tier 2, extracted with sqlglot from what the gold SQL references.

The two are kept as parallel standards rather than merged, since a method’s ranking is a stronger claim when it survives both. Section 3.3 reports what Tier 2 actually contains, which is where the two tiers diverge and why. Section 3.4 closes with the verified statistics for the corpus and both tiers.

3.1 The Spider Dataset

All experiments in this thesis are conducted on Spider (Yu et al., 2018), a large-scale, cross-domain benchmark for Text-to-SQL parsing. Spider spans 200 databases across 138 distinct domains, with no database shared across the training, development, and test splits, a design intended to test generalization to unseen schemas rather than memorization of a fixed database (Yu et al., 2018; Qin et al., 2022).

Reported example counts vary slightly across papers depending on the Spider release cited. Taniguchi et al. (2021), whose annotations this thesis draws on directly for its Tier 1 gold standard, report 8,625 training examples, 1,034 development examples, and 2,147 test examples. This thesis’s own data-loading code loads 7,000 training examples over 140 databases and 1,034 development examples over 20 databases, for 166 confirmed database schemas in total.

The 7,000 figure corresponds to Spider’s `train_spider.json` file specifically. It excludes a further ~1,600 examples that some releases bundle in from five earlier single-domain datasets, which this thesis’s pipeline does not load. Each query in Spider is further labeled with one of four difficulty levels - easy, medium, hard, and extra hard - based on the number of SQL components, selections, and conditions it contains (Yu et al., 2018).

Spider’s schemas vary enormously in size. Across the 166 schemas this thesis’s pipeline loads, the mean is 5.28 tables and 27.13 columns per database (SD 3.87 / 30.80), ranging from three

two-table databases up to `baseball_1`'s 26 tables and 352 columns by a wide margin the largest schema in the benchmark. Figure 3.1 shows the full distribution: in both panels the mean sits well right of the median, and for columns the standard deviation exceeds the mean.

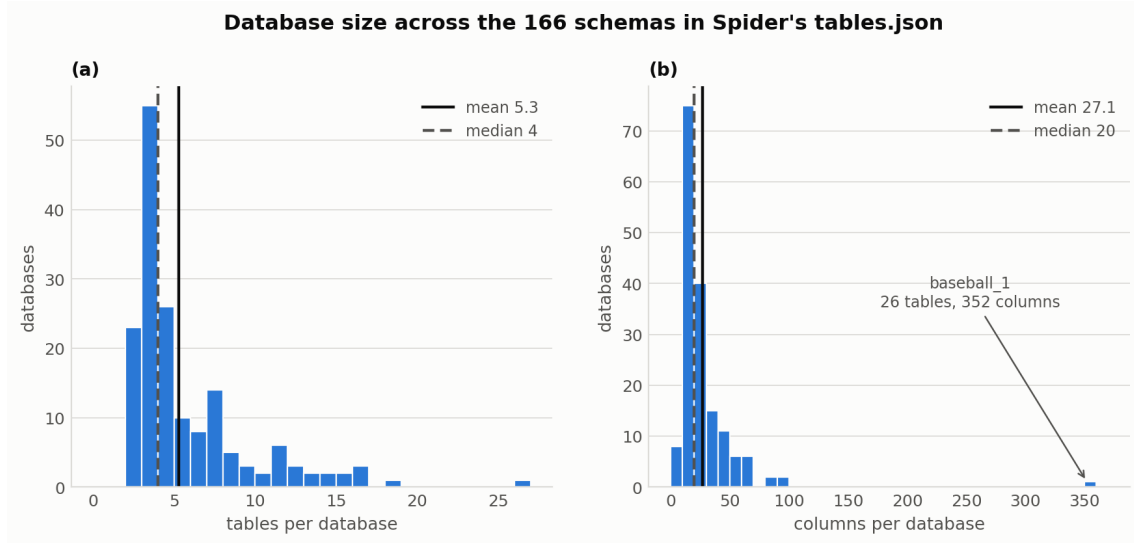


Figure 3.1: Database size across the 166 schemas in Spider's tables.json. tables (a) and columns (b) per database. The mean (solid rule) sits right of the median (dashed) in both panels; `baseball_1` is the extreme at 26 tables and 352 columns.

The questions themselves are short. Dev questions average 12.43 words, with a median of 12 and a range of 4 to 28, and the difficulty labels are weighted towards the middle: 446 of the 1,034 dev questions (43.1%) are medium, against 248 easy (24.0%), 174 hard (16.8%) and 166 extra hard (16.1%). Figure 3.2 gives both distributions. They matter later, Section 3.3 shows that the gap between the two gold tiers is almost entirely a property of the harder end of this distribution.

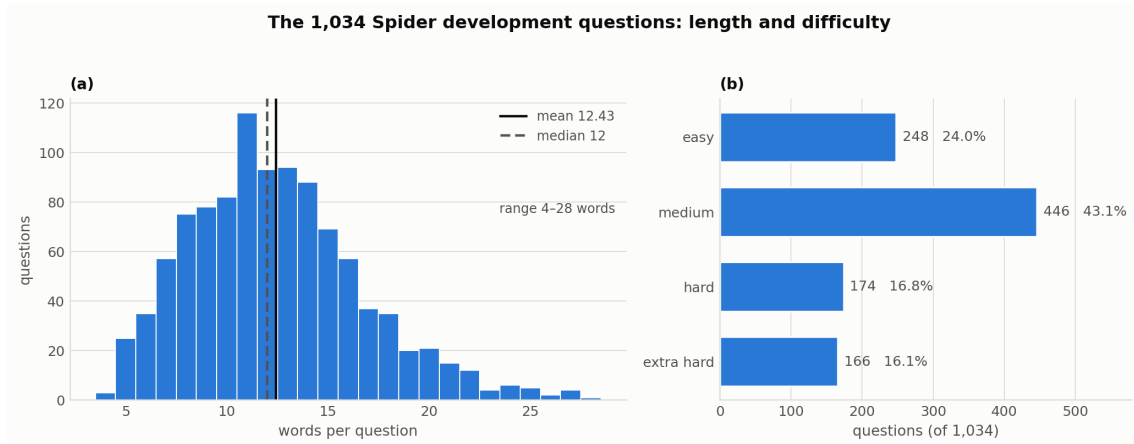


Figure 3.2: The 1,034 Spider development questions. (a) Length in words, which is short and tightly concentrated mean 12.43, median 12, range 4 to 28. (b) The Spider hardness label, which is weighted towards medium.

At the SQL-structure level, the queries this thesis's gold standard is built from are dominated by simple filtering. 46.2% of dev queries contain a WHERE clause, 26.2% a GROUP BY and 22.3% an ORDER BY, with smaller shares carrying a HAVING clause (7.3%), a LIMIT (17.7%), or a set operation - UNION, INTERSECT or EXCEPT at 7.4% combined.

This distribution is why join-heavy, multi-table reasoning is concentrated in a minority of questions, and it is exactly that minority where the largest gaps exist. Figure 3.3 gives the per-clause shares, and Figure 3.4 the pairwise co-occurrences behind them, notably that WHERE and GROUP BY appear together on only 18 questions.

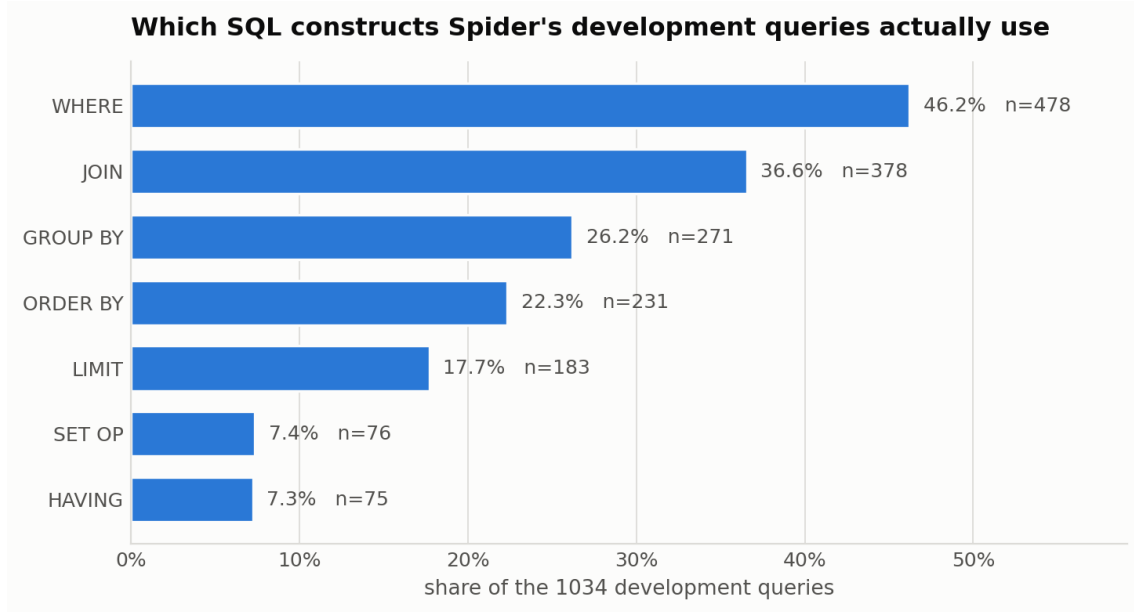


Figure 3.3: Share of the 1,034 development queries carrying each SQL construct. SET OP folds UNION, INTERSECT and EXCEPT together; presence is read from Spider’s own parsed sql dict.

A constraint that shapes this thesis’s entire gold-standard design follows directly from how Spider is distributed. Gold SQL queries are publicly released for the training and development splits, but not for the test split, which is withheld for leaderboard evaluation (Taniguchi et al., 2021).

Since this thesis’s programmatically extracted gold standard (Tier 2, Section 3.2.2) is derived by parsing gold SQL text, it can only be constructed where that SQL is actually available, the training and development splits. This thesis therefore evaluates exclusively on Spider’s training and development data. No result in this thesis should be read as measured against Spider’s official hidden test set or leaderboard.

Each Spider database schema is distributed with two parallel naming conventions for every table and column: an original name, corresponding to the actual identifier used in the underlying SQL database (for example, a column literally called uid), and a semantically friendlier name intended to be more human-readable (airline_id) (Li et al., 2023).

This thesis matches all predictions and gold links against original names, Spider’s `column_names_original` field rather than the friendlier alternative, for two reasons. First, it is the harder and more realistic target: a deployed system queries a real database, which exposes only its actual column identifiers, not a curated paraphrase.

Second, and more specific to this thesis’s design, it keeps both gold-standard tiers on the same representational footing. Tier 2 is extracted by parsing SQL text, which necessarily references a database’s actual column names, so scoring Tier 1 against original names as well ensures both tiers describe correctness in the same terms rather than silently comparing apples to oranges. With the corpus and its naming conventions fixed, the question becomes what counts as a correct link within it.

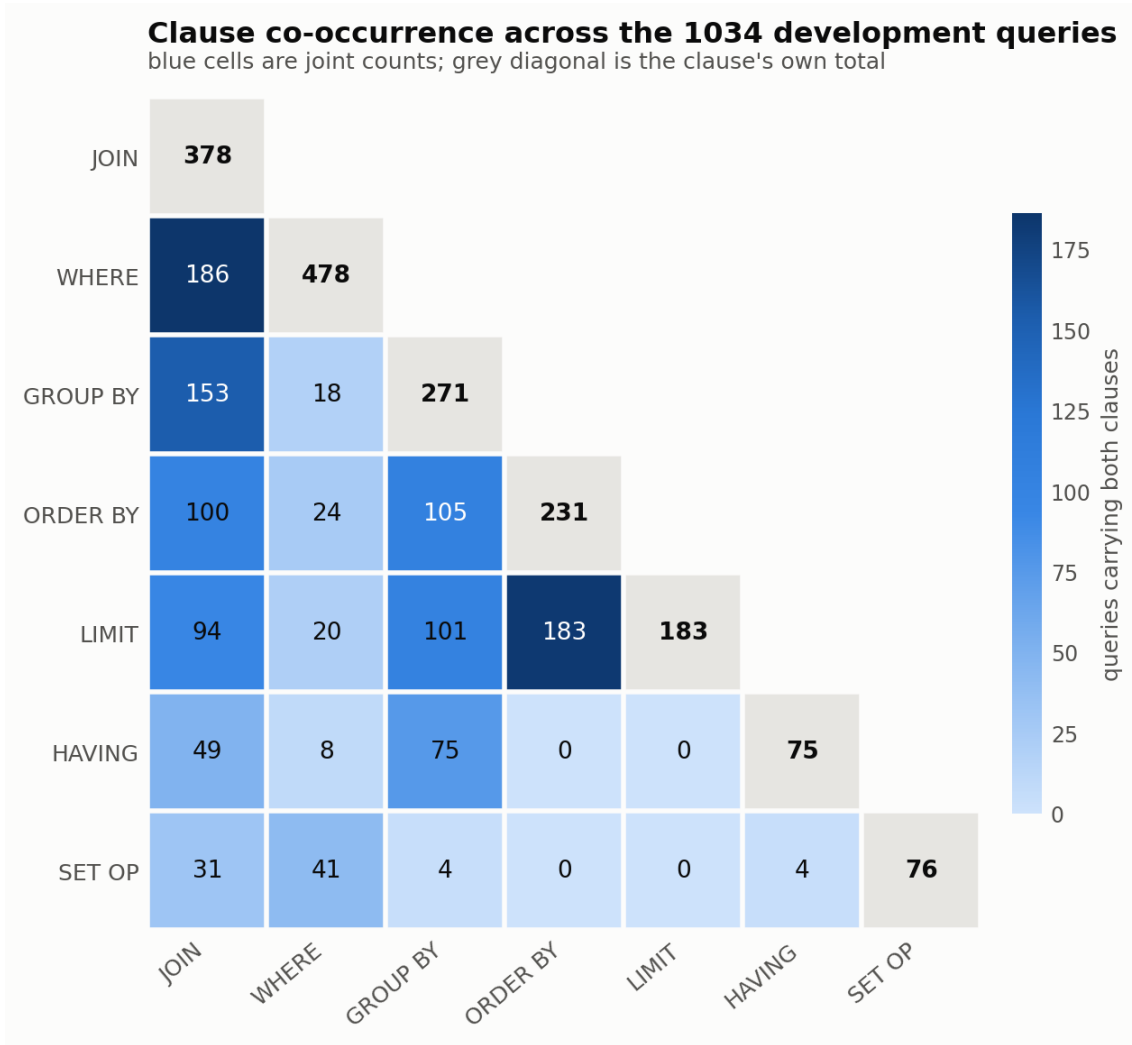


Figure 3.4: Clause co-occurrence across the 1,034 development queries. Blue cells are joint counts, the grey diagonal each clause’s own total; WHERE and GROUP BY co-occur on only 18 queries.

3.2 Gold-Link Annotation Approaches

This thesis constructs its gold standard from two independent sources, referred to throughout as Tier 1 and Tier 2. They are deliberately kept as two separate, parallel gold standards rather than merged into a single set of links. Each represents a different, defensible answer to the question “what counts as a correct schema link?”, and every method is evaluated against both.

Figure 3.5 shows what that difference amounts to on a single question, before either tier is defined formally. The question, its schema and its gold SQL are drawn as three stacked bands; blue arrows run down from the spans a human annotator marked, orange arrows up from the identifiers the gold SQL actually uses. Two columns are reached only from below. They are the join keys, and no correct query can be written without them but the question never names them.

3.2.1 Tier 1 (Mentioned): Human Annotations (Taniguchi et al., 2021)

Tier 1 answers the question “which schema elements does the question’s text actually mention?” It is built from the human annotations released by Taniguchi et al. (2021). Two annotators familiar

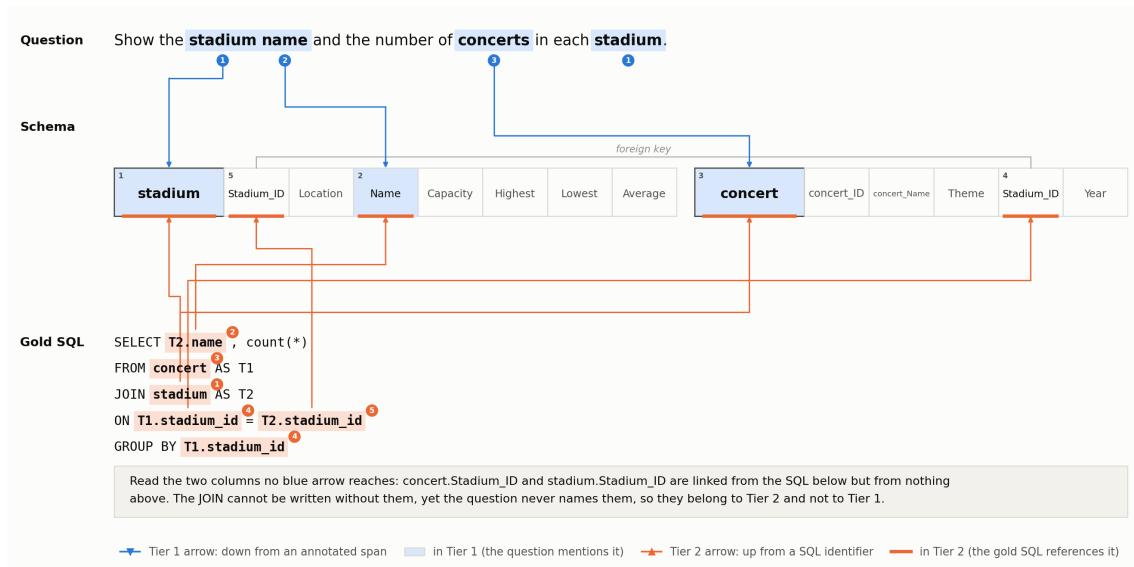


Figure 3.5: Where each tier’s links come from, on development question 22 (concert_singer). Blue arrows descend from the question’s annotated spans Tier 1. Orange arrows ascend from the gold SQL’s identifiers Tier 2. The two columns no blue arrow reaches, concert.Stadium_ID and stadium.Stadium_ID, are the whole reason this thesis keeps two tiers rather than one. Appendix C gives three further worked cases.

with SQL independently labeled character spans of each question in Spider’s development set using the Doccano annotation tool, tagging each span with the specific table or table.column it referred to. Across the full annotated set, the average question carries 2.98 labels (1.18 table references and 1.79 column references, SD 1.23), with a maximum observed count of eight labels on a single question.

Joining Taniguchi et al.’s released annotations to this thesis’s own Spider-indexed data is not entirely mechanical, and the join logic is worth documenting rather than assuming. Taniguchi et al. distribute their 1,034 annotated examples as two files of 517 each, and each annotation carries its own question text rather than a Spider question ID. Matching primarily by exact question-text equality resolves 988 of the 1,034 annotations directly.

The remaining 46 differ from Spider’s own question field by typo fixes, punctuation, or whitespace that Taniguchi et al.’s own normalization introduced. For those 46, this thesis’s data-loading code falls back to an empirically verified id-offset rule: Taniguchi’s own annotation ids run contiguously from 3616 to 4649, and id − 3616 equals the corresponding Spider development row index. The alternative dropping the mismatched examples, or attempting a fuzzy text match that could silently mis-join a question to the wrong schema was rejected.

Every fallback case is logged, and every resolved label is additionally validated against its target schema before being accepted. The join itself fails loudly, raising rather than silently truncating, if more than 1% of labels fail that validation. That ceiling is never approached in practice, since this thesis’s full annotated set resolves successfully.

Two properties of this tier are worth stating explicitly, because they shape how it should be interpreted. First, “mentioned” is a property of the question’s surface text, not of the SQL query’s logic. A table or column that is genuinely required to answer the question but never referred to in its wording a foreign key needed purely to join two tables, for instance will not appear as a Tier 1 label, even though a correct SQL query cannot be written without it.

Second, annotation coverage is limited to Spider’s development split. Taniguchi et al. (2021) did

not annotate the training split, and the Spider test split’s SQL is not public in the first place (Section 3.1).

One implementation detail is worth noting alongside these. Taniguchi et al.’s own paper further re-splits its annotated development data into two internal halves of 517 examples each for its own experiments, a split specific to their paper, and not one this thesis adopts. This thesis’s data-loading code draws on the full annotated development set, and confirming that full coverage against Taniguchi et al.’s public release is one of the verification steps of this thesis’s own gold-standard construction pass.

3.2.2 Tier 2 (All-SQL-Used): sqlglot-Extracted Links

Tier 2 answers a different question: “which schema elements does the gold SQL query actually reference?” Rather than relying on human judgment about a question’s wording, this tier is constructed automatically by parsing each example’s gold SQL query with sqlglot (tobymao, 2024), a Python SQL parser and transpiler.

The walk collects every table and column identifier the parsed query references, whether in its SELECT list, WHERE or HAVING conditions, JOIN ON clauses, or GROUP BY / ORDER BY expressions. Because this extraction operates on SQL text rather than question text, it requires no human annotation effort and can therefore be applied uniformly to both the training and development splits, unlike Tier 1. Figure 3.6 shows the procedure end to end.

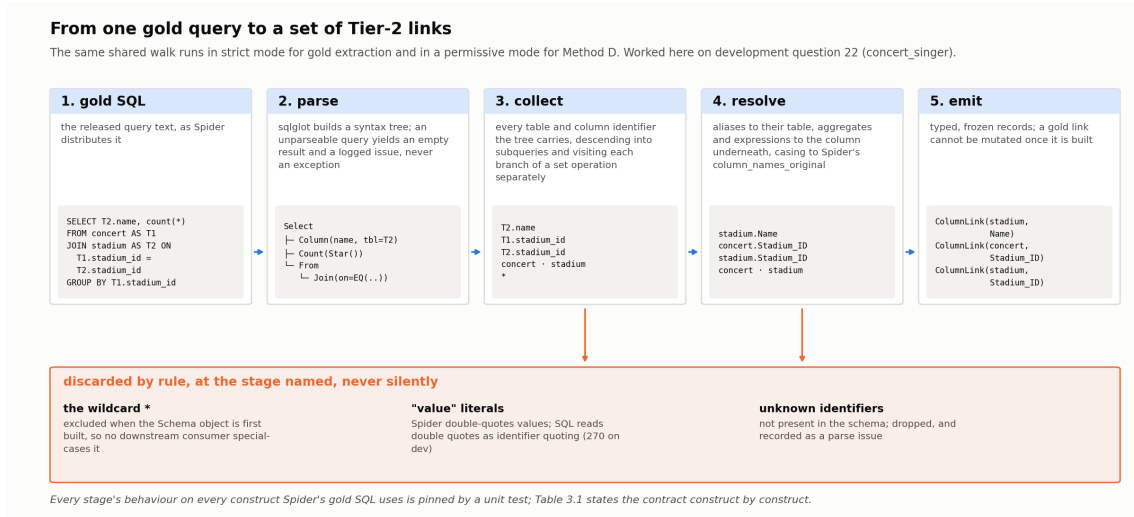


Figure 3.6: How one gold query becomes a set of Tier-2 links, worked on development question 22. The same shared walk runs in a strict mode for gold extraction and in a deliberately more permissive mode for Method D (Section 4.2.4), so the two cannot silently drift apart in how they interpret SQL structure. The reject band lists what the walk discards and at which stage; nothing is dropped without being recorded.

Two implementation conventions, locked prior to construction, keep this extraction consistent across the codebase. Extracted references are resolved to Spider’s column_names_original field, and stored as typed, immutable (frozen) records rather than raw strings or dictionaries, so that a gold link cannot be silently mutated once constructed.

The wildcard column receives the same training-free treatment one level earlier than gold-link extraction. Spider represents SQL’s * as a synthetic column at index 0 of its own column list with a sentinel table index, and this thesis’s schema-parsing code excludes that synthetic column when

it first builds each Schema object. The purpose is specific: no downstream consumer - linker, evaluator, or error-analysis code ever needs to special-case it again.

The wildcard is correspondingly excluded from every extracted gold link and from every method's prediction alike, since it does not name a specific schema element and would otherwise inflate agreement artificially. The parsing logic itself lives in a single shared utility, used in a strict mode for gold extraction and reused in a deliberately more permissive mode by one of the schema-linking methods evaluated in Chapter 4.

Tier 2 is only as trustworthy as the walk that produces it. The walk's behaviour on every construct Spider's gold SQL uses is pinned by unit tests and summarised in Table 3.1; the principle throughout is that a reference resolves to the underlying schema element rather than to its surface form.

Table 3.1: How the shared sqlglot walk resolves each SQL construct into a Tier-2 gold link. Every row is pinned by a unit test; the examples are live output against the `concert_singer` schema.

SQL construct	Example	Tier 2 records
Aggregate	SUM(age), MAX(age)	the inner column
Nested function	MAX(CAST(age AS INT)), SUBSTR(name,1,3)	the inner column
Arithmetic expression	age * 2	the inner column
Output alias	name AS n	the source column, not the alias
Wildcard	SELECT *, COUNT(*)	no column; the table is still recorded
Table alias	FROM singer AS T1	resolved to singer
Self-join	players T1 JOIN players T2	one table; set semantics dedupe
Subquery	in FROM and in IN (...)	descended into
Set operation	UNION, INTERSECT, EXCEPT	each branch visited separately
Common table expression	WITH c AS (...)	CTE name excluded from the table set
Identifier casing	select NAME from SINGER	resolved to Spider's original casing
Unknown identifier	SELECT nope FROM singer	dropped, recorded as a parse issue
Unparseable SQL		empty result plus a parse issue; never raises

Run over the corpus, the extraction resolves all 1,034 development queries to 2,809 columns without leaving a single schema reference unresolved. The 270 identifiers it declines are without exception double-quoted string literals. SQL reads double quotes as identifier quoting, but Spider uses them for values, and a value is not a schema element value linking is outside this thesis's scope (Section 1.3).

The training split behaves identically, 99.7% of its 1,949 declined identifiers being value literals of the same kind. One limitation remains: an unqualified column whose name occurs in more than one table of the same schema is resolved by a schema-wide candidate search that can attribute it to the wrong owner, a case 68 development questions (6.6%) could in principle hit.

Because Tier 1 and Tier 2 are constructed from entirely different signals question wording versus SQL logic, they are not expected to agree perfectly. The difference between them is itself informative. A method's rank ordering that is stable across both tiers is a stronger claim than one

that holds only under a single tier’s definition of correctness, which is precisely the comparison this design is intended to make.

Figure 3.7 gives the first measurement of that difference. Tier 2’s column distribution sits a full column to the right of Tier 1’s, though the gap is concentrated rather than uniform: 47% of questions have no Tier-2-only column at all.

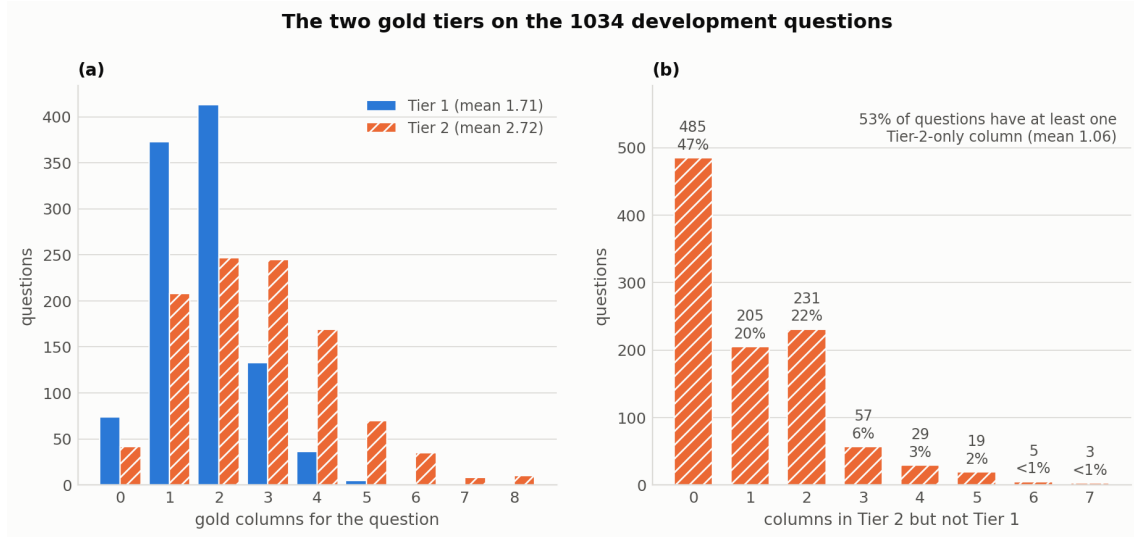


Figure 3.7: Gold columns per development question. (a) Tier 2’s distribution sits a full column right of Tier 1’s; (b) the Tier-2-only surplus, absent on 47% of questions but reaching 7 on a few.

3.3 What Tier 2 Contains

Section 3.2.2 establishes how Tier 2 is built. Because it is a dataset this thesis constructs rather than inherits, its composition deserves reporting in its own right, the more so because Chapter 5’s central finding turns on which parts of it the methods fail to recover.

The starting point is how far the two tiers overlap at all. Pooled across the development split, 1,709 of Tier 1’s 1,767 columns are also in Tier 2, leaving just 58 that are not; Tier 2 adds a further 1,100 on top. At the table level the picture is entirely different: 1,477 links are shared, with only 33 unique to Tier 1 and 88 to Tier 2. Figure 3.8 draws both.

That asymmetry is the first substantive result of the chapter: the choice of tier barely matters for tables, and matters a great deal for columns. The rest of this section is concerned with the 1,100 columns of panel (a).

Every Tier-2 column carries, from the same sqlglot walk that produced it, the set of SQL clauses in which it appears. Table 3.2 reports the resulting composition of the 2,809 development columns, and of the 1,100 of them that Tier 1 does not contain.

Three things follow, and each one bears on a claim made later in this thesis.

First, Tier 2 is not merely Tier 1 plus join keys, and the difference is measurable. Join-only columns are 71.4% of the surplus, not all of it. A further 54 columns appear in a JOIN ON and somewhere else, bringing the share that touches a join at all to 839 (76.3%). That leaves 261 columns (23.7%) that never appear in a join at any point, 127 used only in a WHERE, 60 only in a SELECT, 21 only in an ORDER BY.

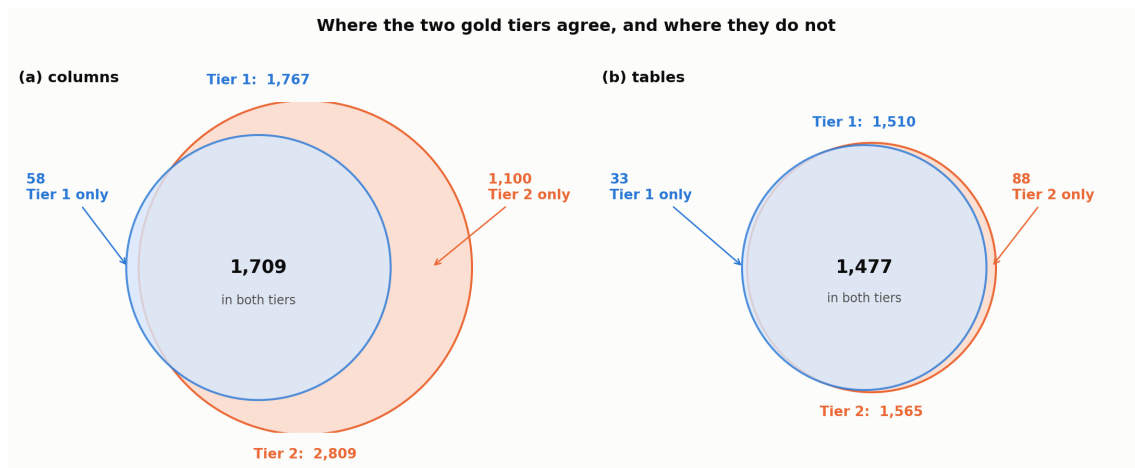


Figure 3.8: Where the two gold tiers agree, and where they do not, pooled over the 1,034 development questions. Circle areas are proportional to link counts and both panels share one scale. (a) At the column level Tier 1 is almost a subset of Tier 2, which adds 1,100 columns of its own. (b) At the table level the two are nearly the same set. The tier gap is therefore a column-level phenomenon, not a table-level one.

Table 3.2: Tier-2 development columns by the clause they appear in. A column counted under “multiple clauses” is used in more than one, most often a column both selected and grouped by. The right-hand pair restricts the same breakdown to columns absent from Tier 1.

Clause role	All Tier-2 (n)	All Tier-2 (%)	Absent from Tier 1 (n)	Absent from Tier 1 (%)
JOIN ON only	823	29.3	785	71.4
SELECT only	983	35.0	60	5.5
WHERE only	484	17.2	127	11.5
ORDER BY only	106	3.8	21	1.9
GROUP BY only	6	0.2	2	0.2
HAVING only	5	0.2	0	0.0
multiple clauses	402	14.3	105	9.5
Total	2,809	100	1,100	100

What Tier 2 records, by the clause a column appears in

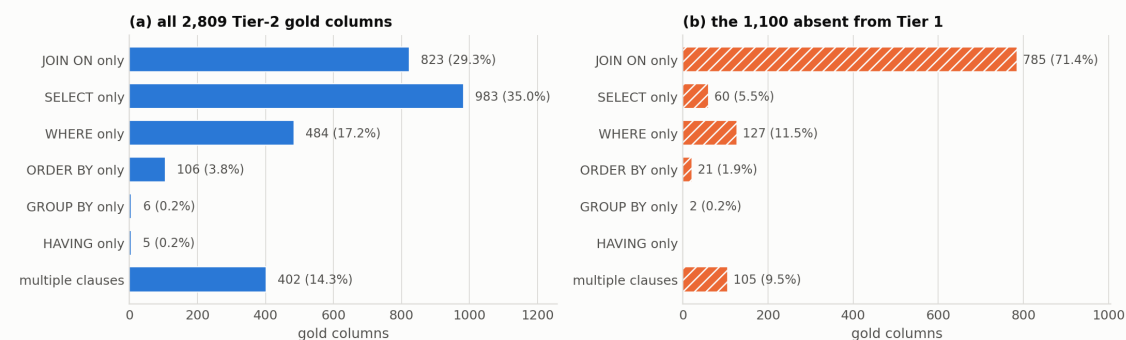


Figure 3.9: The same two distributions. Panel (a) is all 2,809 Tier-2 development columns; panel (b) is the 1,100 that Tier 1 does not record.

These are ordinary, non-structural columns that the gold SQL genuinely requires and that the human annotation simply did not mark. The distinction matters because the two groups have different causes. The join-only columns reflect a definitional difference between the tiers: Tier 1 records what a question mentions, and questions do not mention foreign keys. The 261 reflect incomplete annotation. Only the first is a property of the tier design; the second is a property of the annotation, and it is why Section 5.5 forbids using the set difference Tier 2 minus Tier 1 as a proxy for “join omission”.

Second, join keys are a substantial fraction of Tier 2 overall, not a marginal one. 823 of 2,809 development columns 29.3%, exist in the gold SQL for no purpose other than joining. Close to a third of what Tier 2 asks a linker to find is therefore invisible in the question’s own wording.

This is the quantity that makes the Tier-1/Tier-2 divergence of Section 5.3 as large as it is. It explains why a method optimised against question wording (Method C) and a method optimised against a real query (Method D) can be thirty points apart on one tier and level on the other.

Third, the divergence is concentrated, not spread evenly. 485 of the 1,034 development questions (46.9%) have no Tier-2-only column at all, for nearly half the corpus, the two tiers agree exactly on columns. Of the remainder, 392 questions (37.9%) contribute at least one join-only surplus column and 232 (22.4%) at least one ordinary-clause surplus column.

The concentration follows Spider’s own difficulty axis closely. Averaged per question, the Tier-2-only surplus is 0.15 columns on easy queries, 1.00 on medium, 1.41 on hard and 2.23 on extra hard, a fifteen-fold spread across the four levels, rising monotonically. Figure 3.10 plots it.

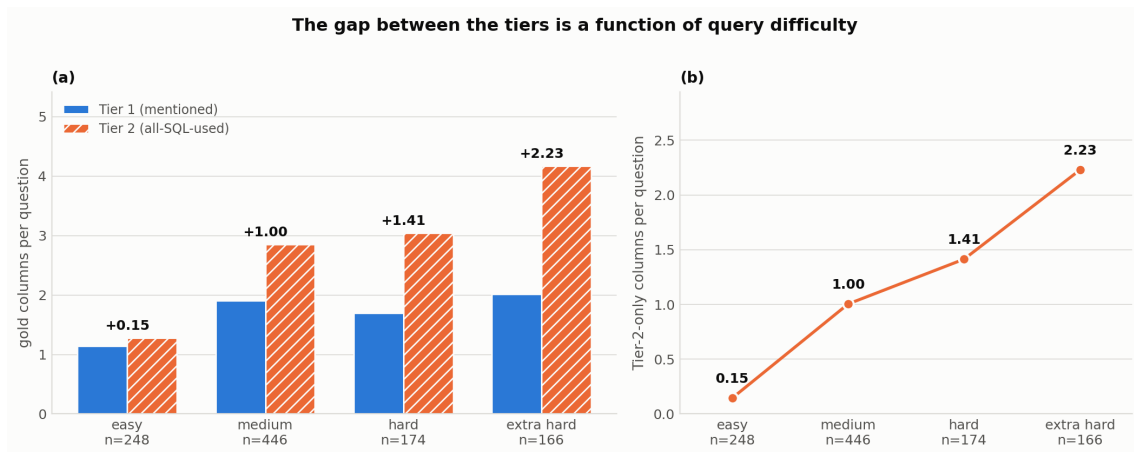


Figure 3.10: Gold columns per development question by Spider hardness. (a) Both tiers, side by side, with the per-question Tier-2-only surplus annotated above each pair. (b) That surplus alone. Tier 1 is close to flat across the four levels while Tier 2 more than triples, so the gap between them is almost entirely a function of query difficulty.

The tier gap is therefore a property of a specific minority of harder, multi-table questions rather than a uniform offset. That is consistent with the clause distribution of Section 3.1, and it is worth carrying into Chapter 5: the hardness breakdowns in Tables 5.2 and 5.3 are measured against a gold standard whose own demands grow with hardness.

The asymmetry runs in the other direction too, though far more weakly. 58 columns across 55 questions appear in Tier 1 but are never referenced by the gold SQL, elements a human annotator marked as mentioned that no correct query touches. Figure 3.11 shows one such case. At the table level, Tier 2 adds 88 tables Tier 1 lacks (join bridges, overwhelmingly) and Tier 1 holds 33 that Tier 2 lacks.

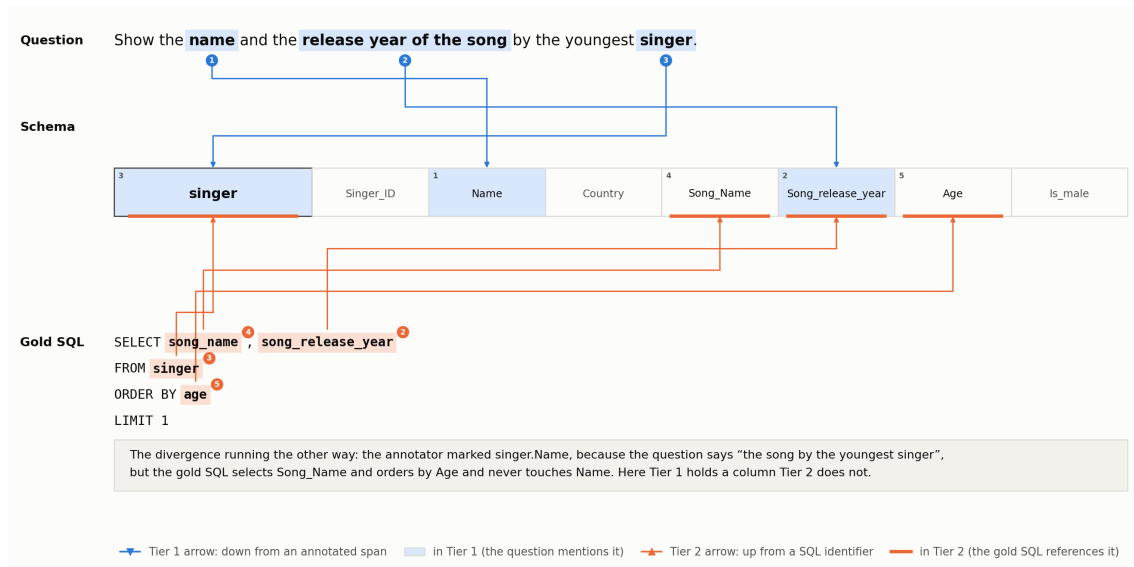


Figure 3.11: The reverse asymmetry, on development question 6. The annotator marked `singer.Name` as mentioned, but the gold SQL selects `Song_Name` and orders by `Age`, and never touches `Name` at all. A method that recovers exactly what the query needs is still scored wrong against Tier 1 here.

These 58 columns are small in number but not in consequence. Section 5.5 shows they become the single largest error source for the strongest methods, since a method that has correctly recovered everything the SQL needs can still be scored wrong against them.

3.4 Dataset Statistics and Splits

This section consolidates every dataset and gold-standard statistic this thesis has directly verified. Table 3.3 covers the Spider corpus as this thesis’s own pipeline actually loads it; Table 3.4 covers the two gold-standard tiers built from it.

Table 3.4 turns to the gold standard itself. One entry needs an explanation rather than a bare number. Taniguchi et al. (2021) own paper reports 3,077 raw annotation labels (1,223 table, 1,854 column), but this thesis’s own loaded and schema-validated Tier-1 set totals 3,277 links (1,510 table, 1,767 column), a different number from the same underlying annotations, not a competing or corrected one.

The two mechanical steps that produce the difference both follow directly from the loading logic documented in Section 3.2.1. First, any table implied by a labelled column but never separately labelled as its own span is added to that question’s table set, which is why the table count rises (1,223 to 1,510) without any new information being added. Second, labels are deduplicated and validated against each question’s actual schema before being kept, which is why the column count falls slightly instead (1,854 to 1,767). Both figures describe the same 1,034 annotated questions; only the convention for counting a table differs.

With both gold-standard tiers, their composition and their canonical representation now defined, Chapter 4 turns to the methodology: the formal problem definition, the six schema linking methods under comparison, the shared linker protocol they all implement, and the evaluation metrics computed against the gold standards defined here.

Table 3.3: Spider corpus statistics, as this thesis’s own pipeline loads and computes them. “166 databases” is train + dev only; the 34 remaining databases belong to Spider’s held-out test split, which this thesis never touches (Section 3.1).

Quantity	Value
Databases (full Spider benchmark)	200
Databases (train + dev only, this thesis’s pipeline)	166
Domains	138
Training examples (this thesis’s pipeline)	7,000
Training examples (Taniguchi et al.’s release)	8,625
Development examples	1,034
Test examples (not used - SQL not public)	2,147
Mean tables per database (SD)	5.28 (3.87)
Mean columns per database (SD)	27.13 (30.80)
Largest schema	baseball_1 - 26 tables, 352 columns
Mean question length, dev (words / chars)	12.43 / 68.05
Dev hardness: easy / medium / hard / extra	248 / 446 / 174 / 166
Dev hardness, as a share of 1,034	24.0% / 43.1% / 16.8% / 16.1%

Table 3.4: Gold-standard statistics for both tiers. *See the preceding paragraph: this figure differs from Taniguchi et al.’s own reported count by construction, not by disagreement or error. The two gap rows are per-question set differences, Tier 2 minus Tier 1; $1.06 \times 1,034 = 1,100$, the surplus reported in Section 3.3.

Quantity	Value
Tier 1 inter-annotator agreement	$\kappa = 0.764$; F1 = 87.5
Tier 1 labels, Taniguchi et al.’s own count (table + column)	3,077 (1,223 + 1,854)
Tier 1 links, this thesis’s loaded set, dev (table + column)	3,277 (1,510 + 1,767)*
Tier 1 gold mean, dev (tables / columns per question)	1.46 / 1.71
Tier 2 gold total, dev (tables / columns)	1,565 / 2,809
Tier 2 gold mean, dev (tables / columns per question)	1.51 / 2.72
Tier 2 gold mean, train (tables / columns per question)	1.55 / 2.88
Tier 1–Tier 2 gap, dev (mean extra tables / columns)	0.09 / 1.06
Tier 1–Tier 2 gap, dev (max extra tables / columns, single question)	3 / 7

Chapter 4

Methodology

4.1 Problem Formulation

Following the notation established in Chapters 1 to 3, let Q denote a natural language question and $S = (T, C, R)$ a database schema, where T is a set of tables, C a set of columns, and R a set of foreign-key relations between columns. A schema linking method is a function $f: (Q, S) \rightarrow \hat{L}$, where $\hat{L} \subseteq T \cup C$ is a predicted link set: a set of table identifiers and table.column identifiers, expressed in terms of Spider’s `column_names_original` field (Section 3.1) and excluding the wildcard column (Section 3.2.2).

This formulation mirrors the structure of both gold-standard tiers defined in Chapter 3. In Tier 1 (L_1), human annotators label spans of Q with the table or table.column they refer to; in Tier 2 (L_2), sqlglot extracts every table and column referenced by the gold SQL query. Predictions and gold links are therefore always compared like for like.

This thesis evaluates six methods, drawn from the four families reviewed in Chapter 2: lexical (Method A, f_A), embedding-based (Method B, f_B), three LLM-based variants distinguished by reasoning direction (Method C forward, f_C ; Method D backward, f_D ; Method E bidirectional, f_E), and graph-based (Method G, f_G). All six implement the same abstract interface (Section 4.3.1), so that a single evaluator can score every method against both L_1 and L_2 without method-specific scoring logic.

4.2 Schema Linking Methods Under Comparison

The six methods are presented in increasing order of the machinery they require, and they divide on two axes. The first is what signal drives the decision: surface string overlap for Method A, vector similarity for Method B, a language model’s judgement for Methods C, D and G, and the schema’s own foreign-key structure for the graph half of Method G. The second is what the method is asked to produce: Methods A, B, C and G name schema elements directly, whereas Method D names none, drafts a SQL query, and has its links recovered by parsing that query.

Two of the six are compositions rather than new machinery. Method E is the set union of Methods C and D and makes no model call of its own. Method G pairs a single model call with a classical path search, so that the model supplies judgement and the algorithm supplies the structural bookkeeping it cannot hallucinate. The sections that follow take each method in turn.

4.2.1 Method A - Lexical Matching

Method A asks whether the words of the question literally appear in the names of the schema. It is the cheapest linker conceivable no training, no model, no API call and it follows the surface string-matching tradition of IRNet’s type-vector heuristics (Guo et al., 2019) and the n-gram annotator benchmarked by Lei et al. (2020); it is included here to establish the floor that any semantically-aware method must clear.

Method A checks every table and column independently against the question, and predicts an element if any one of three strategies fires. All three operate on normalized text: the question is lowercased and every non-alphanumeric character replaced by a space; an element’s name is lowercased and its underscores replaced by spaces, so that `Song_Name` and the question phrase “song name” become the same string.

A token is one maximal run of alphanumeric characters in that normalized text. The question “What is the name and nation of the singer?” yields the token set {what, is, the, name, and, nation, of, singer}, and the element `Song_Name` yields the token sequence (song, name). The distinction that matters between the first two strategies is contiguity, not vocabulary.

1. **Substring:** the element’s normalized name occurs as a contiguous substring of the normalized question. `Song_Name` fires on “...the song name of...” but not on “...the name of the song...”, because the two tokens are not adjacent there.
2. **Token overlap:** every token of the element’s name occurs somewhere in the question’s token set, in any order and at any distance. This is the strategy that recovers “...the name of the song...” for `Song_Name`.
3. **Fuzzy:** `rapidfuzz.fuzz.partial_ratio` on the normalized pair clears a tunable `fuzzy_threshold`. Unlike the first two, this strategy tolerates inexact character overlap: `partial_ratio` slides the shorter string along the longer one, finds the best-matching window, and returns the normalized indel (insertion / deletion) similarity of that window on a 0–100 scale. A score of 100 means the name appears verbatim somewhere in the question; lower scores mean a partially overlapping window was the best available match.

A worked example makes the operating point concrete. On dev question 39 (`concert_singer`), “What is the name and nation of the singer who have a song having ‘Hey’ in its name?”, the fuzzy scores against the singer table’s columns are `Name` 100.0, `Song_Name` 80.0, `Singer_ID` 77.8, `Age` 66.7, `Is_male` 61.5, `Song_release_year` 47.1 and `Country` 42.9.

At `fuzzy_threshold = 70`, three of the seven fire. The column the question actually means by “nation” is `Country`, and at 42.9 it is the furthest of all seven from firing; loosening the threshold enough to catch it would first admit four others. This is the lexical-mismatch failure mode of Section 5.5. Meanwhile `Singer_ID`, which the question never asks for, fires at 77.8 purely because the word “singer” appears. The threshold’s looseness therefore does not buy the missing column; it buys extra ones, which is the mechanism behind Method A’s column precision of 0.403 reported in Section 5.2.

Where more than one strategy fires, the reported strategy follows a fixed precedence: substring, then token, then fuzzy, with substring and token both scored at 100 and fuzzy at its own `partial_ratio` score. The recorded strategy is kept for per-element diagnostics and does not affect the prediction itself.

Tables and columns are evaluated completely independently. A predicted column is never filtered by whether its parent table was also predicted, which keeps this baseline measuring the rawest

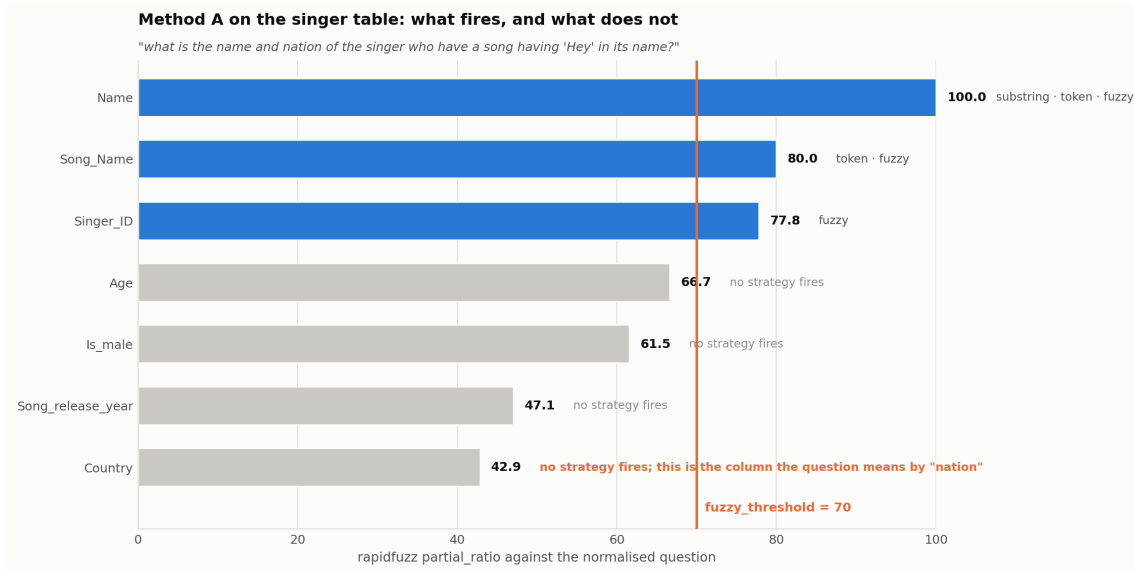


Figure 4.1: Method A on development question 39. Each of the singer table’s seven columns is shown with its `partial_ratio` score against the normalized question, and with which of the three strategies fires. Name clears on all three; Song_Name clears on token overlap but not substring, its two tokens not being adjacent in the question; Singer_ID clears on fuzzy alone. Country, the column the question means by “nation”, scores 42.9 and is the furthest of the seven from the threshold.

possible string-matching signal rather than a filtered, more considered one. That is exactly the lower-bound role this method plays in the comparison (Section 2.3.1).

The single free parameter, `fuzzy_threshold`, was tuned by grid search over 40 to 95 on the seeded 1,000-example train subset, selecting the value that maximizes macro table F1 with column F1 as a tie-breaker. Tables were chosen as the driving signal because Method G’s graph search builds on table-level predictions specifically.

The winner, `threshold = 70`, sits at an interior maximum rather than on the edge of the search space: table F1 falls away in both directions, to 0.745 at 65 and 0.709 at 60 as the match is loosened, and to 0.747 at 75 and 0.653 at 95 as it is tightened (Table 4.3, Figure 4.6). Sweeping below the initial range was not part of the original tuning pass and was added specifically to check that the `argmax` was not an artifact of where the range began; it is not.

Algorithm 4.1: Method A (lexical)

```

input : question Q, schema S, fuzzy threshold t
output: predicted tables and columns

q_norm <- lowercase(Q); every non-alphanumeric character -> space; collapse spaces
q_tokens <- split(q_norm) on non-alphanumeric characters

function MATCH(name):
  n <- lowercase(name); '_' -> space; collapse spaces
  if n is a contiguous substring of q_norm : return ("substring", 100)
  if every token of n occurs in q_tokens : return ("token", 100)
  s <- partial_ratio(n, q_norm)
  if s >= t : return ("fuzzy", s)
  return NONE

for each table T in S:
  m <- MATCH(T.name); if m != NONE: predict T; record diagnostic (T, m)

```

```

for each column C of T:
  m <- MATCH(C.name); if m != NONE: predict (T, C); record diagnostic (T, C, m)

// Columns are never filtered by whether their parent table was predicted:
// this baseline measures the raw string-matching signal (Section 2.3.1).

```

4.2.2 Method B - Embedding-Based Matching

Method B replaces literal overlap with semantic similarity: the question and every schema element are embedded into one vector space by a pre-trained sentence encoder, and an element is kept when its vector is close enough to the question's. It follows the ranking-based family of Li et al. (2023) and Glass et al. (2025), but deliberately with an off-the-shelf encoder rather than a fine-tuned cross-encoder (Section 2.3.2), so that what is measured is what a training-free semantic method buys over string matching.

Method B encodes both the question and every schema element with BAAI/bge-small-en-v1.5 (Xiao et al., 2023), a 384-dimensional English sentence encoder from the BGE / C-Pack family, pinned to a specific HuggingFace commit, so that a re-run cannot silently pick up a re-uploaded checkpoint. The model is used entirely off the shelf: no fine-tuning, no task-specific head, no training data from Spider (Section 2.3.2).

What is embedded is not the raw identifier but a short-rendered phrase, and the rendering rules are fixed in advance: a table is rendered as its human-readable Spider name, and a column as "{column name} of {table name}" so the column `singer.Song_Name` is encoded as the phrase "song name of singer" rather than as the bare token `Song_Name`. Spider's human-readable names are used here specifically because a sentence encoder is trained on natural-language phrases and handles them better than snake_case identifiers; the predictions the method returns are nonetheless mapped back to `column_names_original`, keeping Method B on the same representational footing as every other method and both gold tiers (Section 3.1).

All vectors are L2-normalised at encode time, so cosine similarity reduces to a plain dot product and no separate normalization step is needed at scoring time. Every database's schema vectors are encoded once and cached to disk as a compressed array file keyed by a hash of the model name, the pinned revision, a rendering-rule version tag, and the schema's own metadata; changing any one of those four invalidates the cache automatically, which is what allows the 400-point tuning grid of Section 4.4.1 to re-score without ever re-encoding.

An element is predicted if and only if it clears two independent conditions at once: it is among the `top_k` highest-scoring elements of its kind (table or column) for that schema, and its score separately clears a threshold. Tables and columns are scored with entirely separate `top_k`/threshold pairs, giving four free parameters in total.

The two conditions cannot both bind at once. Because the elements are sorted by score, an element below the threshold at rank `k` implies every later element is below it as well, so the rule is equivalent to keeping the first $\min(\text{top_k}, \text{number above threshold})$ elements. Over the development split, `top_k` is the binding condition on 964 of the 1,034 questions (93.2%) at the column level and the threshold on only 32 (3.1%); at the table level the split is 645 (62.4%) against 97 (9.4%). Figure 4.2 shows one question in detail.

All four parameters were tuned together by an exhaustive grid search: `table_top_k` $\in \{1, 2, 3, 5\}$, `table_threshold` $\in \{0.30, 0.40, 0.50, 0.60, 0.70\}$, `column_top_k` $\in \{3, 5, 8, 12\}$, `column_threshold` $\in \{0.30, 0.40, 0.50, 0.60, 0.70\}$, giving 400 configurations. The search ran on the same seeded 1,000-example train subset used for Method A, scored against sqlglot Tier-2 train gold.

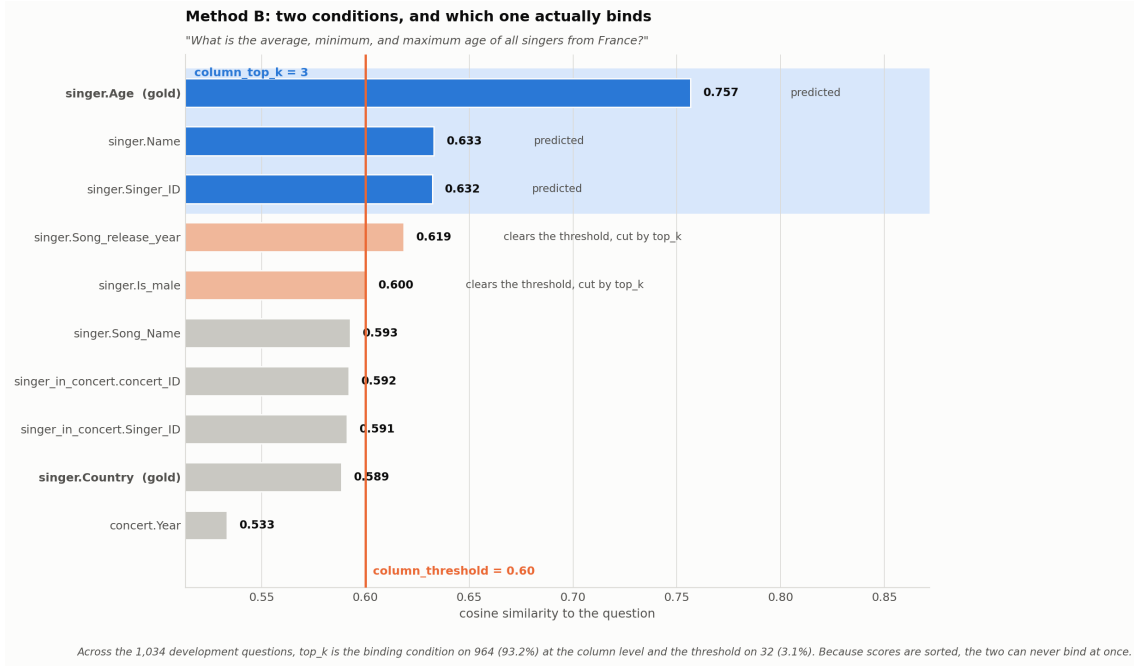


Figure 4.2: Method B’s two conditions on one development question. Every column of the schema is ranked by cosine similarity to the question. The vertical rule is `column_threshold = 0.60` and the shaded band is the `column_top_k = 3` cut. Two columns clear the threshold but fall outside the top three and are dropped, while `singer.Country`, which the gold SQL requires, ranks ninth and is missed by both conditions at once.

It was kept fast by precomputing the full question-schema similarity matrix once and re-applying each of the 400 top-k/threshold masks against it, rather than re-encoding anything per configuration. Selection was by $\text{argmax mean_f1} = (\text{table_f1} + \text{column_f1}) / 2$, ties broken by higher mean recall. The winning configuration, `table_top_k = 2`, `table_threshold = 0.50`, `column_top_k = 3`, `column_threshold = 0.60`, scored `table F1 = 0.693`, `column F1 = 0.550`, `mean_f1 = 0.621` and `mean_recall = 0.752` on the tuning subset itself. Section 4.4.1 and Figure 4.7 report the shape of the grid around that point.

It is worth stating explicitly what Method B does not see, since it is the structural difference between this method and the LLM-based ones that follow. Method C is given the whole schema at once, as a single rendered document of CREATE TABLE statements including foreign-key constraints, and can therefore reason about elements in relation to one another.

Method B never sees a schema as a whole. Each table and each column is embedded in isolation, from its own short rendered phrase and nothing else, and is then scored against the question independently of every other element. No foreign key, no sibling column, and no table context enters the vector. The only place the schema exists as an object rather than as a bag of independent elements is the ranking step, where top_k is applied per database so elements compete for a slot, but never inform each other’s score. Method B is, in this precise sense, a retrieval method rather than a reasoning one.

Algorithm 4.2: Method B (embedding)

```
input : question Q, schema S, parameters (k_T, th_T) for tables, (k_C, th_C) for columns
output: predicted tables and columns

// once per schema, cached to disk under a hash of model + revision + rendering rules
for each table T in S : v_T <- ENCODE(readable_name(T))
```

```

for each column C of T: v_C <- ENCODE(readable_name(C) + " of " + readable_name(T))
// ENCODE returns L2-normalised vectors, so cosine similarity is a dot product

v_Q <- ENCODE(Q)

for level in {tables, columns} with parameters (k, th):
scores <- { e -> v_e . v_Q for every element e at this level }
candidates <- the k highest-scoring elements of this level
for each e in candidates:
if scores[e] >= th: predict e

// The two conditions are independent: k caps the count regardless of how many
// clear th, and th can still drop an element that made the top-k cut.

```

4.2.3 Method C - LLM-Forward

Method C gives an LLM the question and the entire schema in one prompt and asks directly which tables and columns are needed. This is the most direct use of an LLM as a linker; reasoning forward from question to schema and follows the few-shot ranking approach of Dong et al. (2023) C3.

Method C prompts Claude Haiku 4.5 with the question and the full rendered schema, CREATE TABLE statements with inline FOREIGN KEY constraints, using original (DB-canonical) identifiers throughout, since that is the form that appears in real SQL and in Spider’s own gold-link JSON and asks it to return the tables and columns needed, without drafting SQL. The locked system prompt (forward_v1) is reproduced verbatim in Appendix B; its operative rule is rule 3 (Table 4.1), which explicitly excludes columns whose only purpose is joining tables together, restricting the model’s own selection to what would appear in SELECT, WHERE, GROUP BY, HAVING, or ORDER BY of the target SQL.

The model is sampled $k = 3$ times at temperature 0.7 and the three parsed samples are aggregated by a vote threshold: union keeps any element named in at least 1 of 3 samples, majority requires $\lceil k/2 \rceil = 2$, and intersection requires all 3. This thesis locks union for its reported results, following the sampling evidence reviewed in Section 2.3.3 that additional samples reliably trade precision for recall.

The vote denominator is always the full $k = 3$, never the count of samples that happened to parse successfully. A sample that fails to parse (invalid JSON) or fails output-schema validation (valid JSON in the wrong shape) contributes an empty vote rather than shrinking the threshold. A parse failure can therefore only ever hurt recall, never inflate it by making the bar easier to clear.

Hallucinated names are a deliberate design choice, not an oversight. Predicted tables and columns are never filtered against the real schema inside Method C itself, specifically so that the hallucination-rate metric reported in Chapter 5 measures the model’s actual raw behavior rather than a version already cleaned up before scoring.

In practice, across all 1,034 dev questions and both gold tiers, Method C produced zero hallucinated tables and zero hallucinated columns. That is an empirical finding about Claude Haiku 4.5 on Spider-scale schemas, not a property the method’s design guarantees. The only nonzero hallucination count in this study belongs to Method D and, by inheritance, Method E: 11 hallucinated column names across 11 of the 1,034 dev questions, and zero hallucinated tables, a per-element rate of 0.003. Section 5.2 shows that all eleven share a single cause and that none is an invented schema name.

A revised prompt, `forward_v2`, was drafted after inspecting Method C’s worst failures on a 20-example stratified train iteration set: rule 3 was generalized from excluding join-only columns to also excluding whole tables included only because they are topically related to the question rather than actually referenced by the target SQL.

Re-run on the same 20 examples, `forward_v2` did not reproducibly outperform `forward_v1`. Across repeated iteration runs the two versions’ mean-F1 gap (0.005 to 0.008) flipped sign three times, smaller than this 20-example set’s own noise floor, and the specific worst-case pattern that motivated v2 (three of five worst `forward_v1` failures pulling in an extra join-adjacent table or column) reproduced identically under v2 in every run. `forward_v1` was retained as the frozen prompt; `forward_v2` remains registered in the codebase for a future ablation with a larger iteration set rather than being discarded.

A separate, diagnostic self-consistency check, not part of the frozen pipeline compared $k = 1$ against all three $k = 3$ aggregation rules on a 100-question sample of the actual dev-run output already on disk. The result is a genuine limitation worth stating: $k = 3$ union produced identical mean table and column F1 to plain $k = 1$ on this sample (0.9209 / 0.7533 both ways), while $k = 3$ intersection scored marginally higher (0.9242 / 0.7569) and majority matched union on tables and improved column F1 only slightly (0.7553). Union remains the locked, reported aggregation chosen for the recall-oriented reasoning in Section 2.3.3.

Algorithm 4.3: Method C (LLM-forward)

```
input : question Q, schema S, samples k = 3, temperature 0.7, rule = union
output: predicted tables and columns

block <- RENDER(S) // CREATE TABLE statements with inline FOREIGN KEYS,
// using original (DB-canonical) identifiers
votes <- empty counter

repeat k times:
  r <- LLM(system = forward_v1, user = block + two few-shot examples + Q)
  p <- PARSE_JSON(r)
  if p = NULL or not VALID(p, output_schema):
    continue // a failed sample casts an empty vote
  for each element e in p.tables + p.columns: votes[e] <- votes[e] + 1

threshold <- 1 if rule = union | ceil(k/2) if majority | k if intersection
predict every e with votes[e] >= threshold

// The denominator is always the fixed k, never the number of samples that
// parsed, so a parse failure can only lower recall, never lower the bar.
// Predictions are NOT filtered against S: hallucination is measured, not suppressed.
```

4.2.4 Method D - LLM-Backward

Method D never asks about schema elements at all: it asks the model to write the SQL query that answers the question, and then recovers the links by parsing that query. The reasoning runs backward, from a candidate answer to the schema that answer must have used, following Yang et al. (2024) SQL-to-Schema.

Method D asks the same model for a single SQL query answering the question, rather than for schema elements directly. The locked system prompt (`backward_v1`, reproduced verbatim in Appendix B) is markedly simpler than Method C’s: use only tables and columns in the provided schema, return only the SQL with no prose or markdown fences, no inline comments, and resolve any ambiguity in the question toward its most literal reading. No other prompt was ever drafted,

requires at least three failures sharing a pattern before a prompt revision is drafted, and no single failure category reached that bar on Method D’s 20-example iteration set, so `backward_v1` was locked on the first pass.

The generated SQL is parsed by the same shared `sqlglot`-based extractor used to build Tier 2 gold (Section 3.2.2), run in its permissive `strict=False` mode. A hallucinated table or column name in the generated SQL is a valid Method D prediction, not something to filter out, since the whole point of comparing Method D’s predictions to its own SQL is to measure what the model actually wrote rather than a cleaned-up version of it.

Sampling is locked at $k = 1$, temperature 0.0. Rather than instantiate a second LLM client, Method D temporarily overrides the temperature on the same shared client instance Method C uses, swapping it to 0.0 for the duration of each call and restoring it immediately afterward, so that the two methods’ calls are logged through one client and one cost ledger without either affecting the other’s configuration.

Every generated SQL query, whether it parsed cleanly or not, is dumped to a dedicated JSONL file (one line per dev question: question, raw SQL, and any parse issues) specifically to support the error analysis in Chapter 5 it is this file, inspected directly, that traced Method D’s small hallucination rate (Section 5.2) to a single recurring cause: the model writing a syntactically correct aggregate with an alias (`COUNT(*) AS some_alias`), which the SQL walker then reads as if the alias itself were a real column reference.

Algorithm 4.4: Method D (LLM-backward)

```
input : question Q, schema S
output: predicted tables and columns

save t <- client.temperature; client.temperature <- 0.0 // shared client, restored after
sql <- LLM(system = backward_v1, user = RENDER(S) + two few-shot examples + Q)
client.temperature <- t

(tables, columns, issues) <- WALK(sql, S, strict = FALSE) // the Tier-2 walk, permissive
append (Q, sql, issues) to llm_backward_dev_sql.jsonl // for the Chapter 5 analysis
if any issue in issues has kind = parse_error: (tables, columns) <- ( ), ( )
predict tables, columns

// Unresolvable identifiers are kept, not dropped: what is measured is what
// the model actually wrote (Section 4.2.4).
```

4.2.5 Method E - LLM-Bidirectional

Method E takes the set union of Method C’s forward prediction and Method D’s backward one, on the hypothesis that two different reasoning directions fail on different questions. It follows the bidirectional union of Cao et al. (2024) RSL-SQL, whose own ablation reports that the union beats either direction alone for a sufficiently strong model.

Method E makes no LLM calls of its own. It loads Method C’s and Method D’s already-saved canonical dev predictions from disk and returns their set union, table by table and column by column, for every question.

Each prediction’s extra diagnostics record exactly how much each direction contributed and how much they agreed: counts of tables and columns found by forward only, backward only, or both. Figure 4.3 aggregates those diagnostics over the full development split, and shows how one-sided the union is. At the table level the two directions almost entirely agree, with backward contributing just 3 tables the forward direction missed. At the column level backward contributes 661.

Method E’s advantage over Method C alone is therefore a column-level effect almost exclusively, which is what Chapter 5 reports.

If one direction’s prediction is missing for a given question, Method E falls back to the other source alone with a logged warning, and returns an empty prediction with a logged warning only if both are missing.

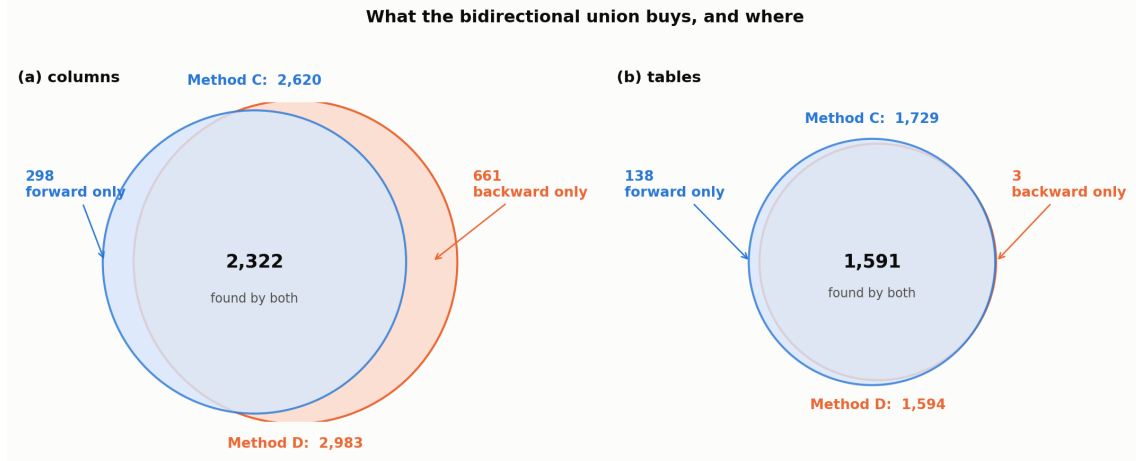


Figure 4.3: What the bidirectional union buys, pooled over the 1,034 development questions. Circle areas are proportional to element counts and both panels share one scale. (a) Columns: the two directions agree on 2,322, forward alone adds 298, and backward alone adds 661. (b) Tables: they agree on 1,591, forward alone adds 138, and backward alone adds 3. Method E is close to Method C at the table level and materially different at the column level.

Algorithm 4.5: Method E (LLM-bidirectional)

```

input : question Q
output: predicted tables and columns

p_C <- Method C's saved prediction for Q
p_D <- Method D's saved prediction for Q

if p_C and p_D are both missing: return the empty prediction (log a warning)
if exactly one is missing : return the other (log a warning)

predict p_C.tables + p_D.tables, p_C.columns + p_D.columns // set union
record | forward only|, | backward only|, | both| as diagnostics

// Method E makes no LLM call of its own; its entire cost is Methods C and D.

```

4.2.6 Method G - Graph-Based

Method G splits the problem in two: an LLM names only the handful of core tables the question is about, and a classical path-search over the schema’s foreign-key graph recovers the connecting tables a correct query would have to join through. It follows Safdarian et al. (2025) SchemaGraph-SQL in leaving the judgement to the model and the structural bookkeeping to an algorithm that cannot hallucinate.

Method G’s single LLM call (`graph_endpoint_v1`, reproduced verbatim in Appendix B) returns two things in one pass: one to three “core” tables, and the specific columns needed. The columns are restricted, by the same SELECT/WHERE-clause criterion as Method C’s rule 3, to columns

that would actually appear in the target SQL rather than columns that only exist to join tables together.

The tables the method actually predicts are then computed by a classical graph algorithm over the schema’s foreign-key graph, built once per schema as an undirected NetworkX graph with one node per table and one edge per pair of tables sharing at least one foreign key. A single core table is returned as is. Two core tables are connected by `shortest_path`, falling back to the two endpoints alone and unconnected if no path exists, logged rather than silently dropped. Three core tables are connected by a greedy Steiner-tree approximation.

Figure 4.4 sets out the two stages and the two independent filters between them; Figure 4.5 works the three-terminal case through in detail.

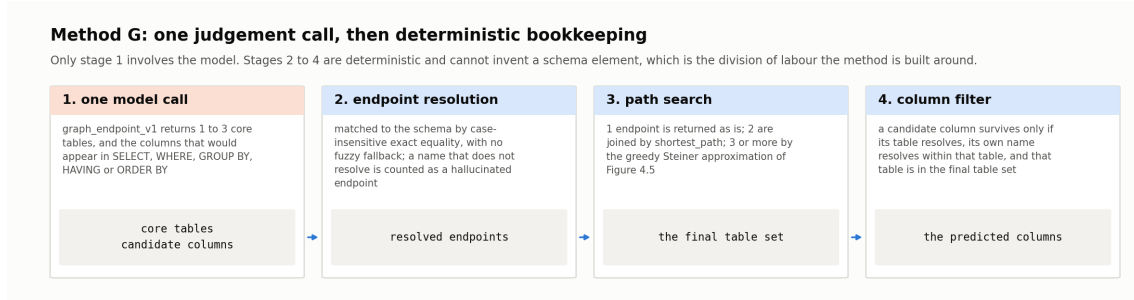


Figure 4.4: Method G’s two stages. One model call names core tables and columns; a path search over the foreign-key graph then decides the final table set, and the predicted columns pass through a second filter that requires each column’s table to have survived the first. The two filters are independent, which is why a column can be entirely real and still be dropped, counted as `columns_dropped_off_path` rather than as a hallucination.

The three-terminal case cannot be solved by a shortest path, because there is in general no single path through three tables. What is wanted is the cheapest subtree of the foreign-key graph that contains all three, a Steiner tree, whose exact computation is NP-hard in the general case.

Method G therefore uses the standard greedy approximation: start the tree at the first terminal; then repeatedly find whichever not-yet-connected terminal is nearest to the tree as it currently stands, compute the shortest path from the tree to that terminal, and merge that path into the tree; stop when every terminal is included. A terminal with no path to the tree at all is added on its own and a warning logged, rather than aborting the prediction. On Spider’s schemas, which average 5.28 tables per database (Table 3.3), the distinction between this approximation and an exact Steiner tree is very largely academic. The approximation is used because it is the standard, cheap and deterministic choice, not because exactness was tested and rejected.

Endpoint resolution is deliberately unforgiving. A raw table name from the LLM is matched against the schema by case-insensitive exact string equality only, with no fuzzy fallback; a name that does not resolve is dropped and counted as a hallucinated endpoint rather than guessed at. In practice, across the full dev run, the endpoint model named zero non-schema tables, so this filter never actually triggered. The design is defensive rather than a response to an observed failure mode.

Predicted columns go through a second, independent filter after the table-level graph computation is already fixed. A column survives only if its own table name resolves, its column name resolves within that table, and, the structural check specific to this method, its resolved table is actually a member of the final predicted table set.

That last condition means a column can be entirely real, a genuine table and a genuine column both present in the schema, and still be dropped, if the LLM named it against a core table that

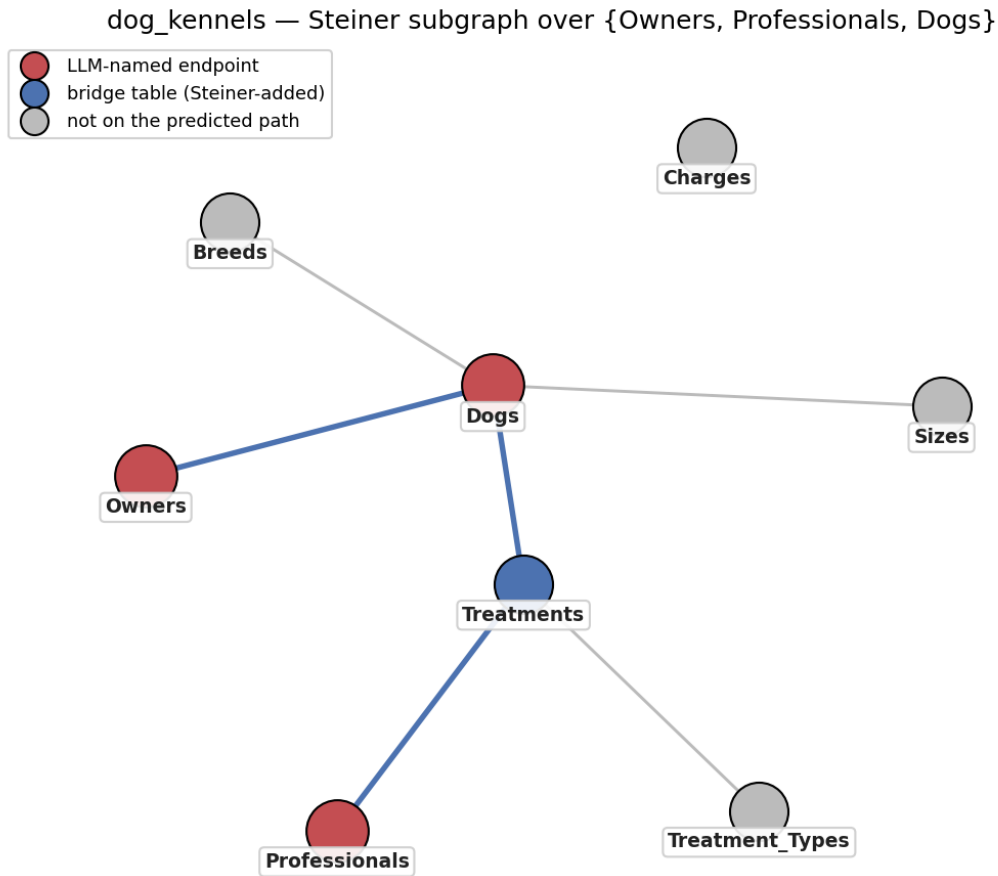


Figure 4.5: The greedy Steiner-tree approximation on `dog_kennels`. The LLM named three core tables (Owners, Professionals, Dogs); no single path connects all three, so the approximation grows a tree from the first terminal outward, and in doing so inserts `Treatments`, a table the model never named and the question never mentions, but without which the three cannot be joined. Grey nodes are the four schema tables the search correctly declined to include.

a two-endpoint shortest-path or three-endpoint Steiner computation did not end up including. Every such drop is counted in the prediction’s diagnostics as `columns_dropped_off_path`, distinct from a genuine hallucination, since the underlying error mode is different: a real column attributed to a table the graph step declined to keep, not a fabricated element.

A revised `graph_endpoint_v2` was drafted after the “wrong endpoint” pattern dominated six of eleven imperfect cases on the 20-example iteration set. The model named either a topically plausible but unreferenced second table as core, or a genuine join-bridge table as core.

The latter directly contradicts `graph_endpoint_v1`’s own rule 2, which instructs the model not to return tables that exist only to join others, on the explicit promise that the graph algorithm will add them (Table 4.1). A join-bridge table named as a core endpoint is therefore not a borderline judgement call by the model but a direct violation of an instruction it was given, which is what made this the failure pattern the v2 revision was drafted to address.

v2 rewrites rule 1 with an explicit “must be needed by the SQL” criterion, and gives rule 2 a concrete, testable definition of “join-only”: no column of the table would appear in `SELECT`, `WHERE`, `GROUP BY`, `HAVING` or `ORDER BY`, and it connects to the rest only via foreign keys. As

with forward_v2, the revision produced no reproducible improvement on the same 20-example set, and graph_endpoint_v1 was retained as the frozen prompt.

Table 4.1: The prompt rules, by method. Full prompt text is reproduced verbatim in Appendix B; this table is the operative summary the body text refers to by rule number.

#	Method C (forward_v1)	Method D (backward_v1)	Method G (graph_endpoint_v1)
1	Use only tables and columns present in the provided schema.	Use only tables and columns present in the provided schema.	Return between 1 and 3 core tables; prefer fewer.
2	Return every table needed and every column needed.	Return only the SQL query - no explanation, no markdown fences, no prose.	Do not include tables that exist only to join others those are added automatically by a graph algorithm.
3	Do not return columns whose only purpose is joining tables, return only columns that would appear in SELECT, WHERE, GROUP BY, HAVING or ORDER BY of the target SQL.	Do not add comments inside the SQL.	For columns, return only those that would appear in SELECT, WHERE, GROUP BY, HAVING or ORDER BY. Not join columns.
4	Output only a JSON object with keys tables and columns.	If the question is ambiguous, choose the most literal interpretation.	Every table and column must come from the provided schema.
5			Output only a JSON object with keys core_tables and columns.

Algorithm 4.6: Method G (graph)

```

input : question Q, schema S
output: predicted tables and columns

G <- graph with one node per table of S, and an edge between two tables
whenever they share at least one foreign key // built once per schema

r <- LLM(system = graph_endpoint_v1, user = RENDER(S) + two few-shot examples + Q)
p <- PARSE_JSON(r)
if p = NULL or not VALID(p, output_schema): return the empty prediction (log a warning)
(core, cols) <- (p.core_tables, p.columns)

endpoints <- [ ]; n_hallucinated <- 0; n_dropped <- 0
for each name in core:
  T <- the table of S whose name equals name case-insensitively // no fuzzy fallback
  if T = NULL: n_hallucinated <- n_hallucinated + 1
  else if T not in endpoints: append T to endpoints

tables <- ( ) if endpoints is empty
(endpoints[0]) if | endpoints| = 1
SHORTEST_PATH(G, e1, e2) if | endpoints| = 2 (both alone if no path, logged)
STEINER(G, endpoints) if | endpoints| >= 3
predict tables

for each (t, c) in cols:
  predict (t, c) if t resolves in S and c resolves in t and t is in tables
  else n_dropped <- n_dropped + 1

```

```

record diagnostic (n_hallucinated, n_dropped)

// Two independent filters: endpoint resolution decides the table set, and the
// column filter then requires the column's table to have survived it.

function STEINER(G, terminals): // greedy Steiner-tree approximation
tree <- { terminals[0] }
for each e in terminals[1..]:
if e is in tree: continue
best <- the shortest of { SHORTEST_PATH(G, v, e) : v in tree }
if best = NULL: tree <- tree + { e } // disconnected: include alone, warn
else : tree <- tree + nodes(best)
return sorted(tree)

```

The two rules set in bold are the ones the rest of this thesis refers to by number. Method C's rule 3 is the instruction that excludes join-only columns, and it is the single design decision most responsible for that method's Tier-2 behavior: it tells the model to withhold exactly the foreign-key columns Tier 2 requires, which is why Method C's Tier-2 column SRR (0.677) falls so far below its Tier-1 figure (0.916). Method G's rule 2, discussed in Section 4.2.6, is the corresponding instruction at the table level: it tells the endpoint model not to name pure join-bridge tables, on the understanding that the graph search will insert them.

4.3 Shared Infrastructure

Six methods that differ this much in machinery are only comparable if everything around them is held fixed. Three components are therefore shared rather than reimplemented per method: the interface every linker satisfies and the record it returns (Section 4.3.1), the SQL walk used both for Tier-2 gold and for Method D's generated queries (Section 4.3.2), and the single API client through which every model call is made, retried, priced and logged (Section 4.3.3). Appendix D maps the codebase onto these roles (Table D.1) and records why each third-party library was chosen (Table D.2).

4.3.1 Linker Protocol (base.py)

All six methods implement a single interface, declared once in `base.py` as a typed protocol with exactly two operations: `predict_one`, which links one question against one schema, and `predict_all`, which links a batch and returns predictions keyed by Spider question id. Both return a `Prediction`, a frozen dataclass holding the database id, a tuple of predicted table names, and a tuple of (table, column) pairs, all in original schema case. The record is immutable and hashable by construction, so a prediction cannot be mutated in place between being produced and being scored.

Four further fields (`table_scores`, `column_scores`, `token_cost`, `extra`) carry per-method diagnostics: the strategy and score behind each lexical hit, the cosine similarity behind each embedding hit, the per-sample vote counts behind each LLM prediction, and the endpoint and path trace behind each graph prediction. All four are deliberately excluded from the saved prediction JSON, so that diagnostics can be as method-specific as they need to be without any of it reaching the evaluator. The evaluator (Section 4.5) is written once against this protocol.

4.3.2 Shared SQL-Parsing Utilities

Section 3.2.2 introduced the shared SQL-parsing utility that underlies Tier 2 gold-link extraction; Sections 4.2.4 and 4.2.6 described its reuse on Method D’s LLM-drafted SQL. The utility exposes a single parsing function with a strict flag. With strict = True, used for gold extraction, it assumes syntactically well-formed SQL and raises on any parse failure, which is appropriate for Spider’s human-authored gold queries. With strict = False, used for Method D, it instead recovers whatever table and column references it can from a query that may be malformed, since an LLM-drafted query is not guaranteed to parse cleanly.

Keeping both modes behind one shared function, rather than maintaining separate parsing logic for gold extraction and for Method D, is what guarantees the two interpret identical SQL syntax identically wherever they overlap. Any measured difference between Method D’s predictions and Tier 2 gold is therefore isolated to genuine differences in SQL content rather than to parser inconsistency.

4.3.3 LLM Client: Retries, Cost Tracking, and Caching

All three LLM-touching methods (C, D, G) share a single client wrapper around the Anthropic API rather than each managing its own HTTP calls. It retries transient failures with exponential backoff up to five attempts: connection errors and rate limits always, server-side 5xx errors conditionally, and client-side 4xx errors never, since a bad request will not succeed on retry.

Every call, successful or not, is logged as one JSONL line: timestamp, model, temperature, a SHA-256 hash of the prompt (for deduplication without storing the prompt text itself), token counts split by input, output, cache-write and cache-read, computed cost, latency, stop reason, and a hash of the response text. Cost is computed from a small, explicit per-model pricing table (Claude Haiku 4.5: \$1.00 / MTok input, \$5.00 / MTok output, \$1.25 / MTok cache write, \$0.10 / MTok cache read).

The complete LLM budget for the thesis was \$11.89 across 6 767 calls, averaging 1 536 input and 44 output tokens per call. Prompt caching was not enabled for the reported runs, so this is an unoptimized upper bound.

Table 4.2: LLM cost by phase (Claude Haiku 4.5).

Phase	Cost (USD)	Calls	Avg in-tok	Avg out-tok
C - prompt iteration	0.25	120	1 866	46.9
C - dev run (k = 3)	6.33	3 577	1 539	45.9
D - prompt iteration	0.16	80	1 832	43.9
D - dev run	3.22	1 946	1 472	36.2
G - dev run	1.93	1 044	1 581	52.8
Total	11.89	6 767	1 536	44.2

4.4 Threshold Tuning, Few-Shot Selection, and Prompt Iteration

4.4.1 Method A and B: Threshold and Grid Tuning

Methods A and B’s free parameters (Sections 4.2.1 and 4.2.2) are both tuned exclusively on a fixed 1,000-example subset of Spider’s training split, sampled with a fixed random seed (42) for reproducibility, and scored against Tier 2 gold. Tier 2 is the only standard available on train:

Taniguchi et al. (2021) annotations cover the development split only, and this thesis never tunes against development data.

To restate the parameters here: Method A has exactly one, `fuzzy_threshold`, the 0 to 100 cut-off above which a `partial_ratio` match counts as a hit, its two other strategies being exact and having nothing to tune. Method B has four, applied as two independent conditions at each of two levels: `table_top_k` and `table_threshold` for tables, `column_top_k` and `column_threshold` for columns, where `top_k` caps how many elements may be predicted and `threshold` sets how similar an element must be to be predicted at all. That is five free parameters in total across the two methods. The three LLM-based methods and the graph method have none of this kind, their behavior being fixed instead by a locked prompt and a locked sampling budget.

The tuning objective is macro F1, not F6. Method A’s single threshold is chosen by table F1, and Method B’s four-parameter grid by the mean of table and column F1, with ties broken toward higher recall. F6, the recall-weighted F-measure defined in Section 4.5.3 and the metric this thesis treats as the best available proxy for a linker’s downstream usefulness, is deliberately not the tuning target.

The reason is that using it for both selection and reporting would let the same recall-heavy weighting shape both the knob and the yardstick. A threshold tuned to maximize F6 would be rewarded for exactly the over-prediction F6 forgives, and the reported F6 would then partly measure that tuning choice rather than the method. Macro F1 is used for selection precisely because it is the more neutral of the two.

Table 4.3: Method A’s full `fuzzy_threshold` sweep, 1,000-example seeded train subset, scored against Tier 2 gold. The argmax on table F1, which is the tuning objective, is interior at 70, with F1 falling in both directions. Column F1 peaks separately at 80.

Fuzzy_threshold	Table			Column	
	P	R	F1	P	F1
40	0.413	0.984	0.535	0.154	0.243
45	0.488	0.946	0.586	0.180	0.277
50	0.525	0.939	0.614	0.196	0.297
55	0.667	0.882	0.683	0.284	0.385
60	0.724	0.858	0.709	0.322	0.419
65	0.798	0.831	0.745	0.381	0.466
70 (chosen)	0.843	0.795	0.751	0.455	0.486
75	0.863	0.765	0.747	0.494	0.497
80	0.884	0.739	0.739	0.574	0.505
85	0.893	0.713	0.721	0.716	0.488
90	0.908	0.657	0.674	0.765	0.485
95	0.918	0.631	0.653	0.789	0.477

Two features of the sweep are worth drawing out. The first is that the two levels do not agree on an optimum: table F1 peaks at 70 while column F1 peaks at 80, so the choice of which level drives the selection is not a formality. Had columns driven it, the frozen threshold would have been 80, raising Method A’s column precision on the tuning subset from 0.455 to 0.574 and lowering its table recall from 0.795 to 0.739. Tables were chosen because Method G’s graph search consumes table-level predictions specifically, and a baseline tuned to a level no other method depends on would be a weaker point of comparison.

The second is that the sweep is flat near its peak, at 0.745, 0.751 and 0.747 for 65, 70 and 75, so the exact value carries little weight. What the sweep establishes is that Method A’s operating point is not perched on the boundary of an arbitrarily chosen range, which the original 70 to 95 sweep

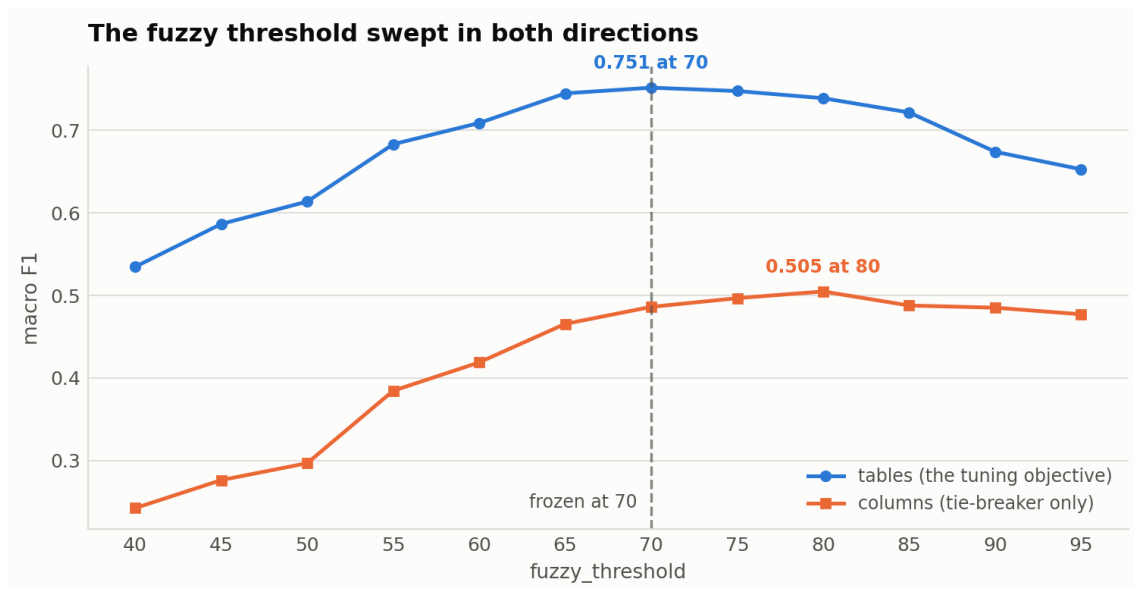


Figure 4.6: Macro F1 against `fuzzy_threshold`, swept from 40 to 95 on the seeded 1,000-example train subset. The table curve, which is the tuning objective, peaks at 70; the column curve peaks separately at 80 and is used only as a tie-breaker.

on its own could not show.

Method B’s 400-point grid ($\text{table_top_k} \times \text{table_threshold} \times \text{column_top_k} \times \text{column_threshold}$) was kept computationally cheap by precomputing the full question-schema cosine-similarity matrix once and re-applying each of the 400 masks against it rather than re-encoding anything per configuration; the full sweep runs in about twenty seconds on CPU. The same protocol, same seed, same subset and same Tier-2-on-train gold govern both methods, so neither is tuned under more favorable conditions than the other.

Figure 4.7 shows the grid around its argmax, and makes two things visible that the winning configuration alone does not. The first is that the four parameters are not equally consequential. Holding the other three at their winning values, `table_threshold` moves mean F1 by 0.189 across its range, `column_top_k` by 0.070, `table_top_k` by 0.035, and `column_threshold` by only 0.026.

The second is that `column_threshold` is very nearly inert. Its value is flat to three decimals at 0.30, 0.40, 0.50 and 0.60, and only costs anything at 0.70. The frozen choice of 0.60 is therefore not a meaningfully tuned value so much as any value below the point at which the threshold starts discarding wanted columns. Read alongside Section 4.2.2’s finding that `top_k` is the binding condition on 93.2% of development questions, the column threshold is doing very little work in this method. As with Method A’s flat peak, this is worth stating rather than leaving the four-parameter grid to imply that all four were equally load bearing.

4.4.2 Few-Shot Example Selection

A few-shot example is a solved instance placed in the prompt ahead of the real question: a schema, a question, and the correct answer in exactly the output shape the method expects back. Its purpose here is narrow and mostly mechanical.

It is not there to teach the model what schema linking is, since Section 5.1’s results show the model already knows, but to fix the form of the answer, so that the output parses as valid JSON

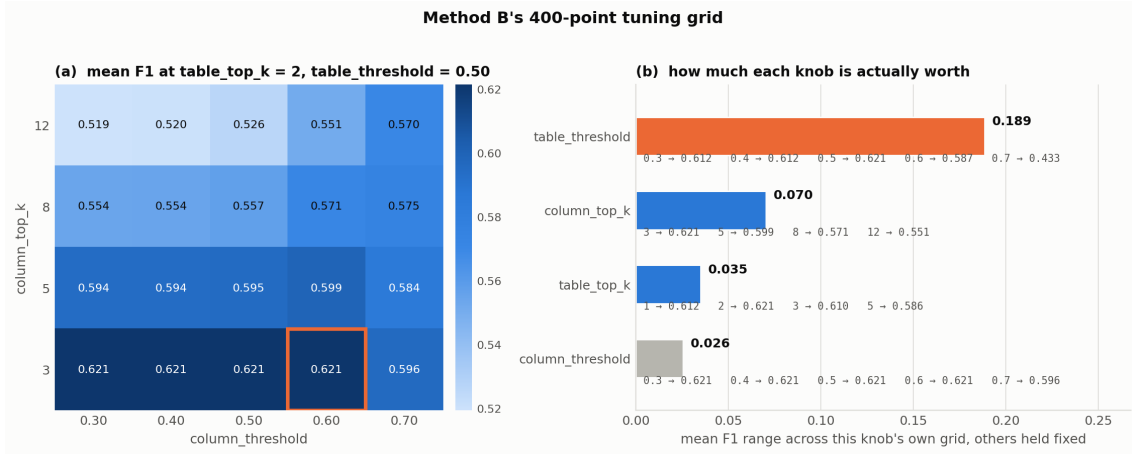


Figure 4.7: Method B’s 400-point tuning grid. (a) Mean F1 across the column grid at the winning table settings, with the selected configuration outlined; the bottom row is flat at 0.621 for four of the five thresholds. (b) Each parameter swept alone with the other three held at their winning values, ordered by how much it moves mean F1. `table_threshold` dominates; `column_threshold` barely registers.

in the expected shape on the first attempt, and to demonstrate the two structural cases that the rules alone state only abstractly: a single-table question and a multi-table joined one.

Because example selection is itself a documented lever on LLM linking performance, with Katsogiannis-Meimarakis et al. (2026) reporting that one demonstration helps but that further ones help inconsistently, the selection procedure is fixed in advance and applied identically to all three LLM-based methods, so that no method can be advantaged by a luckier draw of examples than another.

All three LLM-based methods (C, D, G) use the same two fixed few-shot examples, selected once and reused everywhere rather than chosen per method. This is a deliberate fixed-example strategy, not a retrieval-based or per-question selection. The two examples instantiate two structural patterns fixed in advance: `simple` (a single table, an aggregation, no joins) and `multi_table` (two or more tables joined via `FROM`, with gold columns spanning at least two of them).

Candidates are drawn from Spider train, filtered to Tier-1 gold-linked examples whose `db_id` has not already been used by an earlier pattern, which guarantees the two examples come from different databases, then shuffled with a fixed seed (42), and the first match for each pattern is kept. Every candidate’s gold tables and columns are validated against its own schema before being accepted, so a malformed few-shot example can never silently enter a prompt.

The examples passed are `qid 4913` (`store_product` database, “What is the total number of residents for the districts with the 3 largest areas?”, the `simple` pattern) and `qid 334` (`product_catalog` database, “Which attribute definitions have attribute value 0? Give me the attribute name and attribute ID?”, the `multi_table` pattern).

Each method sees these same two examples reformatted for its own output shape, rather than three separately authored pairs. Method C’s forward format lists tables and columns directly. Method D’s backward format replaces the tables/columns pair with the example’s own gold SQL, drawn straight from Spider train with no LLM involved in generating it. Method G’s graph format renames tables to `core_tables`, relying on the fact that Tier-1 gold already excludes join-bridge-only tables, so every table in a Tier-1 few-shot example is already a valid core or endpoint table with no separate filtering step needed.

4.5 Evaluation Metrics

Every method is scored against both gold-standard tiers (L_1 and L_2) using five metrics, computed per example and aggregated across the evaluation split both ways, macro (the mean of each question’s own precision/recall, weighting every question equally regardless of schema size) and micro (pooled true/false-positive/negative counts across the whole split, weighting every predicted or gold element equally instead).

Figure 4.8 works all five through on a single prediction, so that the definitions that follow can be read against a concrete case.



Figure 4.8: The five metrics on one prediction, Method B on development question 4. The predicted and gold link sets are shown as a partition into true positives, false positives and false negatives, and each metric is then computed from those three counts. SRR is the only one that is not a ratio: it asks a single yes-or-no question, whether the false-negative cell is empty.

4.5.1 Precision, Recall, F1

For a predicted link set $\hat{L}$ and gold link set L on a single example, precision, recall, and F1 are defined in the standard way:

$$\text{Precision} = \frac{|\hat{L} \cap L|}{|\hat{L}|} \quad \text{Recall} = \frac{|\hat{L} \cap L|}{|L|}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

An empty prediction against a non-empty gold scores precision = 1.0 (vacuously, since zero predictions means zero false positives) and recall = 0.0. This is a deliberate departure from scikit-learn’s convention of scoring vacuous precision as 0 with a warning, chosen because a linker that predicts nothing should be penalized entirely through recall rather than having that failure double-counted into precision as well.

Precision penalizes over-prediction, meaning irrelevant tables or columns included; recall penalizes under-prediction, meaning required elements omitted. A missed link is generally more costly than an extra one for downstream SQL generation: an omitted column makes correct generation impossible, whereas an extra one merely adds noise that the model may or may not be misled by. This thesis therefore does not rely on F1 alone, and reports the further metrics below alongside it.

4.5.2 Strict Retrieval Rate (SRR)

Following Cao et al. (2024), SRR measures the fraction of examples for which a prediction achieves perfect recall - every gold element present, with no tolerance for partial credit:

$$\text{SRR} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{L}_i \supseteq L_i]$$

where N is the number of examples in the split and $\mathbf{1}[\cdot]$ is an indicator function. SRR is deliberately harsher than recall: a single missed column zeroes an example's contribution entirely, on the reasoning - shared by Cao et al. (2024) and by Katsogiannis-Meimarakis et al. (2026), who adopt an equivalent definition - that a downstream SQL generator supplied an incomplete schema cannot succeed no matter how small the omission.

4.5.3 F-beta ($\beta = 6$)

Following Glass et al. (2025), this thesis also reports a recall-weighted F-beta score with $\beta = 6$:

$$F_\beta = \frac{(1 + \beta^2) \cdot \text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}$$

so that $F_6 = \frac{37 \cdot \text{Precision} \cdot \text{Recall}}{36 \cdot \text{Precision} + \text{Recall}}$

The $\beta = 6$ weighting follows Glass et al. (2025) own reported finding that, in their end-to-end experiments, F6 tracked downstream SQL generation accuracy more closely than precision or a balanced F1 did; this thesis adopts the same weighting rather than re-deriving its own, so that its results remain comparable to that prior finding rather than introducing a sixth, thesis-specific metric convention into a literature already documented as suffering from too many incompatible ones.

The weighting is on recall, and heavily so: $\beta = 6$ gives $\beta^2 = 36$, meaning recall is weighted thirty-six times as much as precision. The behavior this produce is best seen at the limits. As precision approaches 1, F6 approaches recall; and because precision enters the denominator only through the term $36 \cdot P$, it stops mattering almost entirely once it is large relative to $R/36$. A method must therefore have a catastrophic precision not merely a poor one before F6 penalizes it appreciably.

Method E's Tier-1 columns illustrate the effect exactly. At precision 0.596 and recall 0.972, the balanced F1 is 0.682, which reads as a middling result; the F6 of the same prediction is 0.956, which reads as a near-perfect one. Neither number is wrong. F1 is answering "how clean is this prediction?" and F6 is answering "how likely is it that everything needed is in here?" and the second question is the one a downstream SQL generator actually cares about, which is the reasoning behind Glass et al. (2025) recommendation this thesis follows. It also means F6 must never be read on its own: F6 alongside precision is informative, F6 alone would rank a method that predicts the entire schema at the very top.

Both figures are macro means of the per-question scores rather than the formulas evaluated at the macro precision and recall. For F6 the distinction does not matter, since F6 tracks recall and recall averages linearly. For F1 it does: the formula evaluated at 0.596 and 0.972 returns 0.739, about six points above the reported mean of the per-question values. Figure 4.9 plots both response curves and marks this point on them.

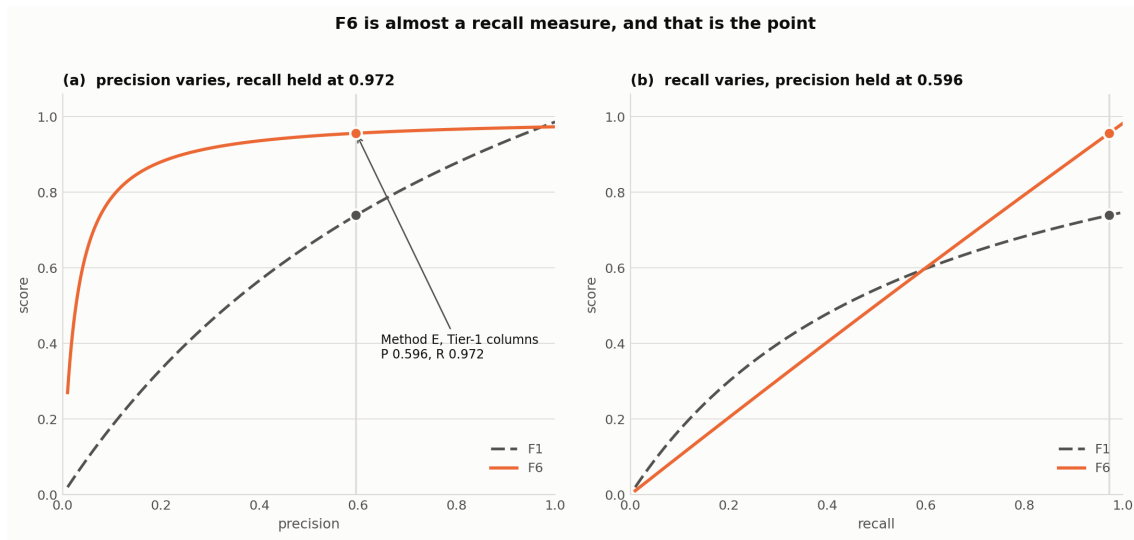


Figure 4.9: How F6 responds, compared with F1. (a) Both scores as precision varies at fixed recall: F1 falls roughly linearly while F6 stays near recall until precision becomes very small. (b) The same at fixed precision as recall varies, where F6 tracks recall almost exactly. The marked point is Method E’s Tier-1 columns, at precision 0.596 and recall 0.972.

4.5.4 Hallucination Rate (HR)

The four metrics above all assume that a predicted element exists in the schema. For the three LLM-touching methods that assumption is not free, so this thesis reports a fifth quantity alongside them. A predicted element is hallucinated when it names a table absent from the schema, or a column absent from the table it is attributed to. The hallucination rate for one question is the count of such elements divided by the total number of elements that prediction contained before any filtering, macro-averaged over the split in the same way as the other metrics.

The evaluator removes hallucinated elements from a prediction before computing precision, recall, F1, F6 and SRR, and counts them only under HR. Without that separation a hallucination would be charged twice once as a false positive and once as a hallucination and the two failure modes are worth keeping distinct: a real-but-unnecessary column is noise a downstream generator can ignore, whereas a nonexistent one produces SQL that cannot execute at all. Methods A, B and G cannot hallucinate by construction, so HR is only ever informative for Methods C, D and E; Section 5.2 reports what it found for each.

4.6 Statistical Testing (McNemar’s Test)

Precision, recall, F1, SRR, and F6 establish how methods rank; they do not by themselves establish whether an observed difference between two methods is larger than what could arise from the particular sample of questions being evaluated on. This thesis therefore supplements its five metrics with McNemar’s test (McNemar, 1947) for eight selected pairwise comparisons between methods, computed separately against each gold-standard tier.

For a pair of methods, each example is scored as a binary success or failure using the same perfect-recall criterion as SRR. An example counts as a success for a method if that method’s prediction fully contains the gold link set, and a failure otherwise. The two methods’ per-example outcomes on the identical set of examples are then arranged in a 2×2 contingency table of the four possible outcome pairs: both succeed, both fail, only the first succeeds, only the second succeeds.

McNemar's test statistic depends only on the two discordant cells, the examples where the methods disagree, and tests the null hypothesis that a disagreement is equally likely to favor either method. A significant result indicates that one method's advantage over the other is unlikely to be an artifact of which particular questions happened to be evaluated. Because the same fixed set of examples is scored by every method, this pairing controls for per-example difficulty in a way that comparing two methods' aggregate metrics alone cannot. That is why this thesis reports both: the five metrics of Section 4.5 for magnitude, and McNemar's test for whether that magnitude is statistically defensible.

Chapter 5

Results

This chapter reports results for all six methods, against both gold-standard tiers, at both the table and column level, macro-averaged over the full $N = 1,034$ Spider development split. That the split is complete confirms, against the concern raised in Section 3.2.1, that this thesis’s data-loading code draws on Taniguchi et al. (2021) full annotated development set.

Total measured LLM spend across the evaluation run was \$11.89 across 6,767 API calls to Claude Haiku 4.5, within the cap discussed in Section 4.3.3.

5.1 Overall Performance Comparison Across Methods

Table 5.1 reports precision, recall, F1, F6, SRR, and per-prediction-element hallucination rate for every method, gold tier, and level. Two headline results anchor the chapter: the best Tier-2 column F1 of any method is 0.939, achieved by Method D (LLM-backward); the best Tier-1 column SRR of any method is 0.951, achieved by Method E (LLM-bidirectional).

Every method is recall-heavy at the column level, which is the correct bias for a filter feeding a downstream generator, where a missed column costs more than an extra one. As Section 5.3 details, the methods reach that bias by very different routes. Figure 5.1 puts every row of the table on one axis pair so those routes are visible before the per-method discussion begins.

5.2 Per-Method Analysis

Method A (lexical) behaves exactly as Section 2.3.1 predicts: it over-predicts substantially, retaining 4.49 columns per question against a Tier-1 gold mean of only 1.71. That caps its column precision at 0.403 even though its recall (0.863) stays competitive.

This is the signature of a method that matches generic English words against real column names with no semantic filter. Figure 5.2 places every method’s prediction budget against both gold means at once, and the ordering is informative in itself: the two methods at the extremes are the weakest (Method A, 4.49) and one of the strongest (Method G, 2.40), so volume alone predicts very little. What separates them is not how much each predicts but how well the budget is spent. Method A’s 4.49 columns capture 86.3% of Tier-1 gold; Method G’s 2.40 capture 95.8%.

Method B (embedding) does not improve on Method A’s table precision. At 0.662 against Method A’s 0.842 it is in fact lower, since embedding similarity is a noisier table-level signal than exact

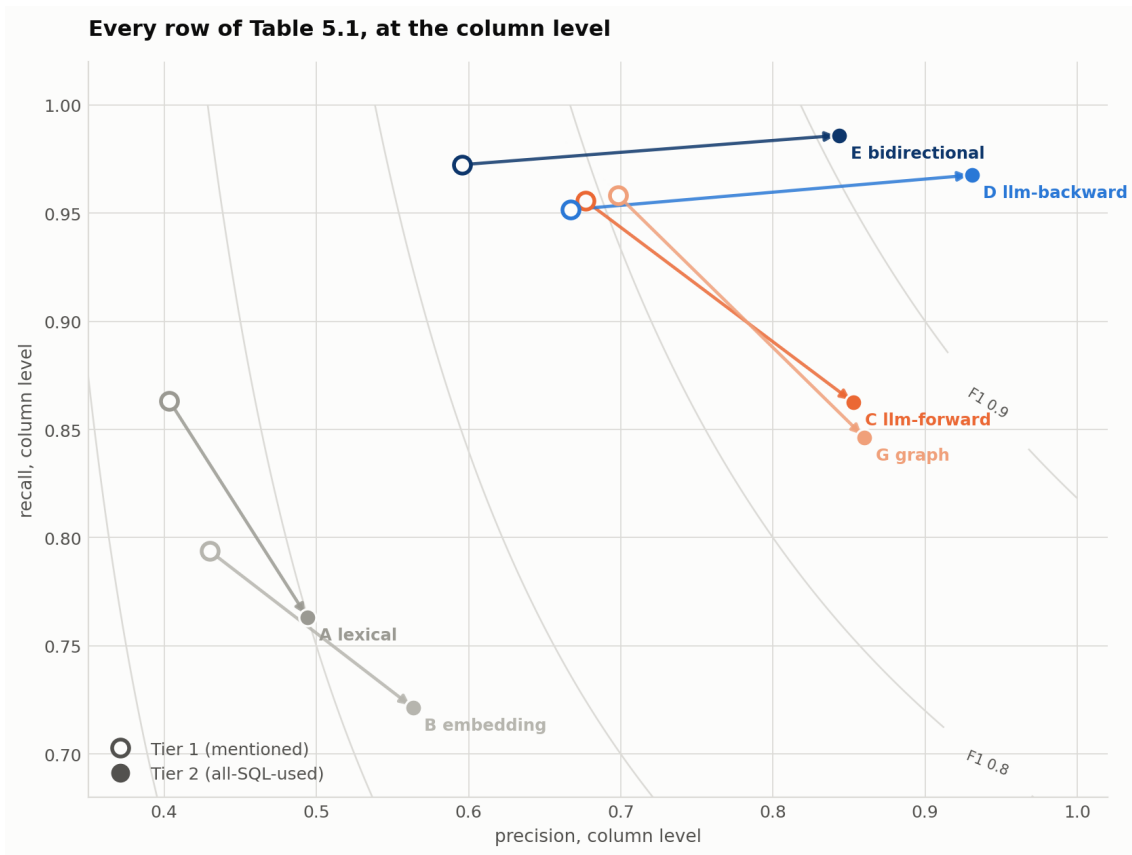


Figure 5.1: Every row of Table 5.1 at the column level, as one point per method and tier. Each arrow runs from a method’s Tier-1 score to its Tier-2 score, and contours mark equal F1. Every arrow points right, because Tier 2’s larger gold turns some of what a method already predicted into a correct link. What separates the methods is the vertical direction: only Methods D and E point up. The other four lose recall, because Tier 2 asks for the join keys they never predicted.

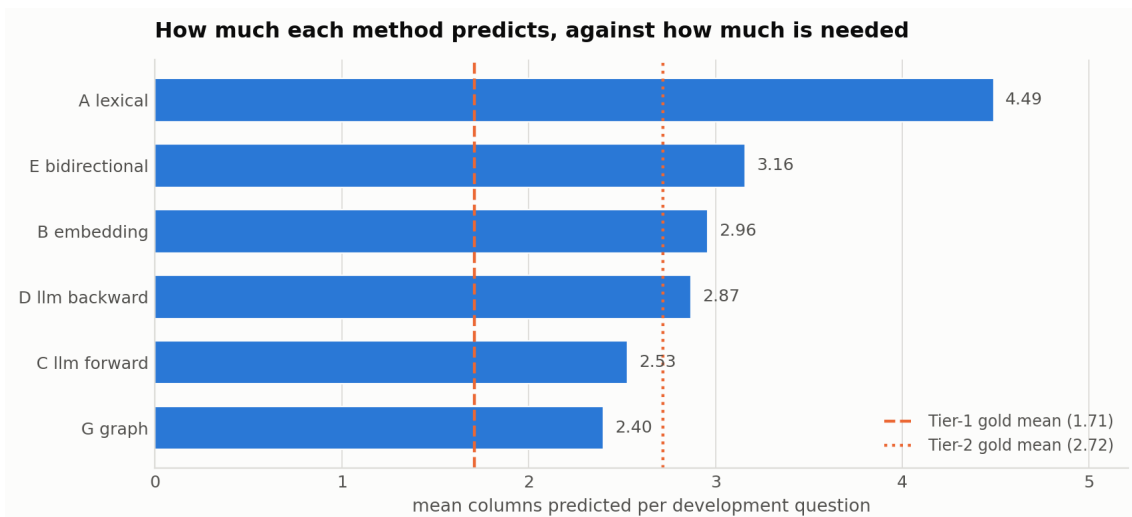


Figure 5.2: Mean columns predicted per development question, by method, against both gold standards’ own means. Method A predicts 4.49 columns where Tier 1 asks for 1.71, the over-prediction behind its column precision of 0.403. Method G is the most economical at 2.40, below even the Tier-2 requirement of 2.72, which is the other side of the join-omission result of Section 5.5.

Table 5.1: Main results, Spider dev, macro average, N = 1,034. HR = per-prediction-element hallucination rate. Tier 1 is the Taniguchi et al. (2021) human annotation; Tier 2 is sqlglot parsed over the gold SQL (Chapter 3).

Method	Tier	Level	P	R	F1	F6	SRR	HR
A lexical	1	tables	0.842	0.785	0.737	0.787	0.715	0.000
		columns	0.403	0.863	0.484	0.837	0.768	0.000
	2	tables	0.850	0.779	0.736	0.780	0.699	0.000
		columns	0.494	0.763	0.528	0.752	0.554	0.000
B embedding	1	tables	0.662	0.885	0.731	0.877	0.791	0.000
		columns	0.430	0.794	0.514	0.776	0.647	0.000
	2	tables	0.678	0.880	0.737	0.873	0.772	0.000
		columns	0.564	0.721	0.573	0.716	0.469*	0.000
C llm forward	1	tables	0.907	0.988	0.932	0.985	0.974	0.000
		columns	0.677	0.956	0.741	0.945	0.916	0.000
	2	tables	0.935	0.998	0.957	0.996	0.997	0.000
		columns	0.853	0.863	0.815	0.862	0.677*	0.000
D llm backward	1	tables	0.949	0.978	0.953	0.977	0.957	0.003
		columns	0.667	0.952	0.739	0.941	0.927	0.003
	2	tables	0.980	0.989	0.980	0.989	0.984	0.003
		columns	0.931	0.968	0.939	0.967	0.923	0.003
E llm bidirectional	1	tables	0.905	0.988	0.931	0.985	0.974	0.003
		columns	0.596	0.972	0.682	0.956	0.951	0.003
	2	tables	0.934	0.998	0.956	0.996	0.997	0.003
		columns	0.844	0.986	0.879	0.981	0.959	0.003
G graph	1	tables	0.929	0.987	0.946	0.985	0.970	0.000
		columns	0.698	0.958	0.757	0.949	0.917	0.000
	2	tables	0.958	0.996	0.970	0.995	0.992	0.000
		columns	0.860	0.846	0.808	0.847	0.653*	0.000

string matching. What it buys instead is table recall (0.885 against 0.785), which is what carries it to the better table SRR discussed in Section 6.1 (0.791 against 0.715).

At the column level it is the weakest method of all under Tier-2 SRR (0.469), the single lowest value in Table 5.1. Off-the-shelf sentence embeddings, without task-specific fine-tuning (Section 4.2.2), struggle most precisely where Tier 2’s SQL-derived definition of correctness diverges furthest from surface-level semantic similarity to the question.

Method C (LLM-forward) and Method G (graph) share a distinctive profile: both are excellent Tier-1 linkers and comparatively mediocre Tier-2 ones. Both reach near-perfect Tier-2 table performance (SRR 0.997 and 0.992 respectively) while their Tier-2 column SRR falls to 0.677 and 0.653. They correctly identify which tables are involved and then drop the specific join keys connecting them, a gap invisible at the table level and stark at the column level.

Both also show zero measured hallucination throughout, which is a structural property rather than a fortunate outcome. Method C selects from the schema it is given rather than generating free text, and Method G’s predictions can only ever be real tables and columns traversed along real foreign-key edges, so neither has a mechanism by which a nonexistent element could enter its output.

That Method G in particular drops join keys deserves to be confronted rather than merely recorded, since joined tables are precisely what a graph-based method exists to handle. The resolution is that Method G’s two outputs are produced by two different mechanisms, and only one of them is a graph.

Its tables come from the foreign-key path search, and there the method performs as its design promises: 0.992 Tier-2 table SRR, statistically indistinguishable from Method C (Section 5.4), with the connecting tables inserted by the path search rather than named by the model. Its

columns, however, come from the same single LLM call that names the core tables, and that call is instructed by rule 3 (Table 4.1) to return only columns appearing in SELECT, WHERE, GROUP BY, HAVING or ORDER BY, which is an instruction to withhold join keys. The graph is never consulted about columns at all.

The method therefore fails at the column level for the same reason Method C does, and it is the prompt rather than the graph that fails. This is a sharper result than a simple weakness would be, because the missing information is demonstrably already present: each foreign-key edge the path search traverses is a pair of columns, and those pairs are exactly the ones the prediction omits. Section 6.3 develops the point closing that loop is the clearest single improvement this study identifies.

Method D (LLM-backward) is structurally immune to the table/column gap that limits Methods C and G under Tier 2. Because its predictions are read directly out of a parsed SQL query (Section 4.2.4) rather than named directly by the model, it achieves 0.923 Tier-2 column SRR and the highest Tier-2 column F1 of any method (0.939).

This robustness carries a small cost: a 0.003 hallucination rate, the only nonzero rate in Table 5.1 alongside Method E, which inherits it. This is not the model misnaming a column, as this thesis initially assumed. The actual cause, confirmed by inspecting every one of the eleven affected dev queries individually, is a single recurring parsing artifact: the model writes a correct SQL aggregate with a COUNT-style alias, and the sqlglot walker lifts that alias as if it were a real column reference on the table being counted.

Every hallucinated name fits this pattern exactly: `concert.concert_count`, `players.win_count`, `Students.enrollment_count`, `Transcript_Contents.transcript_count`, three separate aliases on `TV_Channel` (`num_channels`, `channel_count`, `number_of_TV_Channel`), and `orchestra.format_count` and `orchestra.frequency`. Not one is an arbitrary or unrelated invented name, and all eleven are self-flagged in the run log as an `unknown_column` parse issue (Section 5.5).

A single case shows the whole pattern, and Figure 5.3 works it through. On dev question 26 (`concert_singer`), “Which year has most number of concerts?”, Method D generated a query that is correct, executes, and answers the question. It is a defensible alternative formulation of the gold SQL, which reaches the same answer with an unaliased ORDER BY `count(*)`.

The sqlglot walk, reading identifiers out of that query, resolves the output alias `concert_count` against the only table in scope and records `concert.concert_count`, a name that exists nowhere in the schema. The parse is self-flagged in the run log as `unknown_column`. The remaining ten cases are the same event with a different alias: `players.win_count` (qid 463), `students.enrollment_count` (541, 542), `transcript_contents.transcript_count` (571), three aliases on `tv_channel` (595, 596, 605, 606) and two on `orchestra` (850, 851).

Method D’s 0.003 hallucination rate is therefore best read as an extraction artifact of this thesis’s own SQL walker, not as evidence of the model inventing schema.

The eleven cases were inspected individually because they are few enough to permit it. They are not, however, the extent of the failure inspection in this thesis. The analysis reported in Section 5.5 covers the complete population of 2,107 failure cases across all six methods and both gold tiers, each one resolved to a specific question id, a specific missing or extra element, and a specific cause. Sections 5.5 and 6.4 report those distributions per method; the paragraphs above are concerned only with what the aggregate metrics of Table 5.1 do and do not show.

Method E (LLM-bidirectional) posts the best Tier-1 column SRR (0.951) and the best Tier-2 column SRR (0.959) of any method. It is the only method that leads both tiers at the column level, and it does so by taking the union of Method C’s forward prediction and Method D’s backward one (Section 4.2.5).

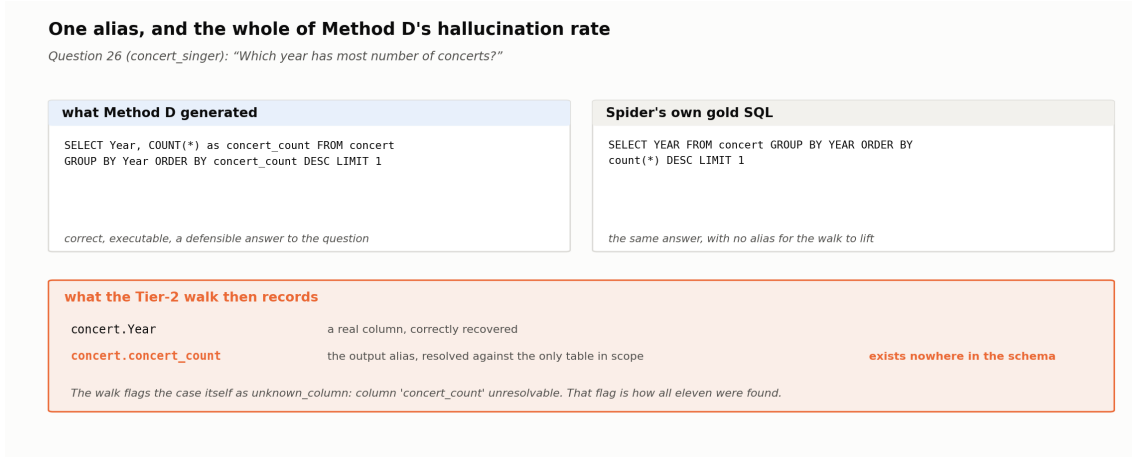


Figure 5.3: The whole of Method D's hallucination rate, on development question 26. The model's SQL is correct and executable; the alias `concert_count` is an output name, not a schema reference, but the walk resolves it against the only table in scope and emits `concert.concert_count`. The walk flags the case itself, as `unknown_column`, which is how all eleven were found.

That union is not indiscriminate. Method E predicts 3.16 columns per question at Tier 1 (precision 0.596, recall 0.972, matching Table 5.1), fewer than Method A's 4.49 though a broader net than Method G's economical 2.40. This is consistent with Section 2.3.3's account of Cao et al. (2024) own finding that a bidirectional union outperforms either direction alone: Method C's Tier-1 strength and Method D's Tier-2 robustness combine rather than cancel.

Method G (graph) is the most economical strong method in Table 5.1: it predicts only 2.40 columns per question, fewer than any other method except the weaker Method A and B baselines, while achieving the best Tier-1 column precision (0.698) and Tier-1 column F1 (0.757) of any method, at zero LLM sampling cost beyond its single core-table-identification call (Section 5.3).

Spider's four difficulty levels (Section 3.1) separate the six methods cleanly. On Tier-1 tables, Method A falls from 0.774 SRR on easy questions to 0.512 on extra-hard, and Method B falls similarly, from 0.948 to 0.536. Both baselines degrade sharply as questions grow more complex, whereas Methods C, E and G are comparatively flat across the same range, holding between 0.928 and 0.992. Difficulty hurts the non-LLM baselines roughly three times as much as it hurts the LLM-touching methods.

Table 5.2 gives the full picture at the column level, where the pattern sharpens. Under Tier 2, Methods C and G collapse specifically on the hardest questions, falling to 0.392 and 0.313 on extra-hard, statistically indistinguishable from the lexical baseline, while Methods D and E hold 0.801 and 0.892. Figure 5.4 plots all four strata. The join-omission failure mode identified in Section 5.5 is concentrated in exactly this stratum: it is not that C and G get worse in general as questions get harder, but that they specifically lose the join columns that harder, more multi-table questions require.

Table 5.2 reports strict recall alone, and the reason is worth stating explicitly rather than leaving to inference: SRR is the only one of this thesis's five metrics that answers the question a downstream generator actually poses, which is not "how good is this prediction?" but "is everything I need in here?" A generator handed a schema subset missing one join key cannot recover, however high that subset's F1.

The two metrics do not merely differ in strictness; on this data they tell different stories, and Table 5.3 shows where. Under strict recall, Methods C and G collapse on extra-hard Tier-2 questions, to 0.392 and 0.313. Under F1 the same predictions score 0.795 and 0.783, barely below their

Table 5.2: Strict recall rate by Spider hardness, column level, both tiers. On extra-hard questions, the join-heaviest stratum Methods C and G fall to 0.392 and 0.313 under Tier 2, statistically indistinguishable from the lexical baseline, while D and E hold 0.801 and 0.892.

Method	Tier 1				Tier 2			
	easy	med	hard	extra	easy	med	hard	extra
A lexical	0.831	0.767	0.770	0.675	0.750	0.552	0.511	0.313
B embedding	0.899	0.599	0.563	0.488	0.867	0.426	0.356	0.108
C llm forward	0.968	0.899	0.908	0.892	0.952	0.646	0.638	0.392
D llm backward	0.960	0.935	0.914	0.873	0.972	0.939	0.925	0.801
E bidirectional	0.980	0.946	0.954	0.916	0.980	0.969	0.971	0.892
G graph	0.976	0.906	0.902	0.873	0.960	0.637	0.580	0.313

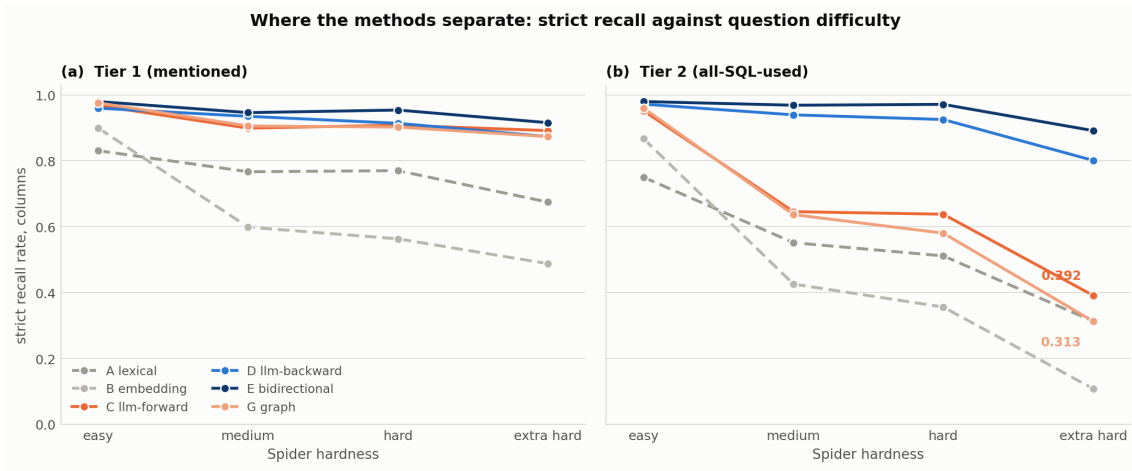


Figure 5.4: Column-level strict recall by Spider hardness, one line per method, Tier 1 in panel (a) and Tier 2 in panel (b). The two panels are the finding: under Tier 1 all four LLM-touching methods stay flat, and under Tier 2 Methods C and G fall away from D and E as the questions get harder.

Table 5.3: Column-level F1 by hardness, both tiers. The same cells as Table 5.2, under a metric that awards partial credit.

Method	Tier 1				Tier 2			
	easy	med	hard	extra	easy	med	hard	extra
A lexical	0.473	0.525	0.444	0.434	0.464	0.572	0.504	0.531
B embedding	0.469	0.561	0.472	0.496	0.494	0.638	0.541	0.551
C llm forward	0.719	0.795	0.705	0.668	0.762	0.858	0.798	0.795
D llm backward	0.892	0.747	0.672	0.558	0.959	0.947	0.919	0.908
E bidirectional	0.708	0.729	0.650	0.552	0.768	0.930	0.890	0.900
G graph	0.722	0.805	0.739	0.701	0.763	0.849	0.790	0.783

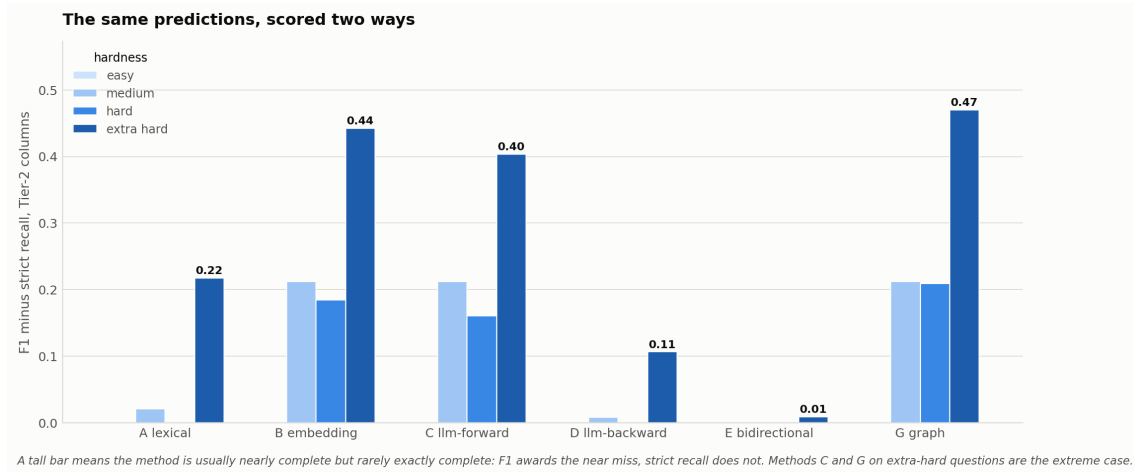


Figure 5.5: How far F1 and strict recall disagree, by method and hardness, on Tier-2 columns. Each bar is F1 minus strict recall for the same predictions. A tall bar means the method is usually nearly complete but rarely exactly complete, which is the near-miss profile of Methods C and G on hard questions.

own easy-question figures, and far above the lexical baseline they were said to be indistinguishable from.

Both readings are correct, and the gap between them is the finding. C and G’s extra-hard predictions are nearly complete, typically missing a single join column out of five or six. That near-miss costs almost nothing under F1 and everything under SRR. Figure 5.5 shows the size of the divergence for every method and stratum: it reaches 0.470 for Method G on extra-hard Tier-2 columns, the largest in the study.

Which metric is right depends on whether a downstream generator can recover from one missing foreign key, and the schema-linking literature reviewed in Section 2.4, Cao et al. (2024) and Katsogiannis-Meimarakis et al. (2026) both, assumes it cannot. Table 5.2 is therefore reported under the assumption that makes the finding meaningful, and Table 5.3 is reported so that the assumption is visible rather than buried.

5.3 Tier 1 vs. Tier 2 Gold-Standard Comparison

Section 3.2 argued that Tier 1 and Tier 2 answer different questions, what a question mentions against what its SQL requires. Table 5.1 shows the distinction is not merely conceptual but empirically decisive.

Under Tier 1 alone, Methods C, E and G look nearly interchangeable, clustering between 0.917 and 0.951 column SRR. Under Tier 2 the same three methods separate sharply: E leads at 0.959, followed by D at 0.923, then C at 0.677 and G at 0.653. That is a spread of roughly thirty points which a single-tier evaluation would never surface. Figure 5.6 draws every method’s move between the two tiers.

This is the practical argument for this thesis’s two-tier design. A method’s Tier-1 ranking is not a reliable predictor of its Tier-2 ranking, because the two tiers reward different capabilities: recognizing what a question is about, against recovering the full join structure a correct query actually needs. Only Method D’s SQL-parsing design, and Method E’s union with it, are robust to both.

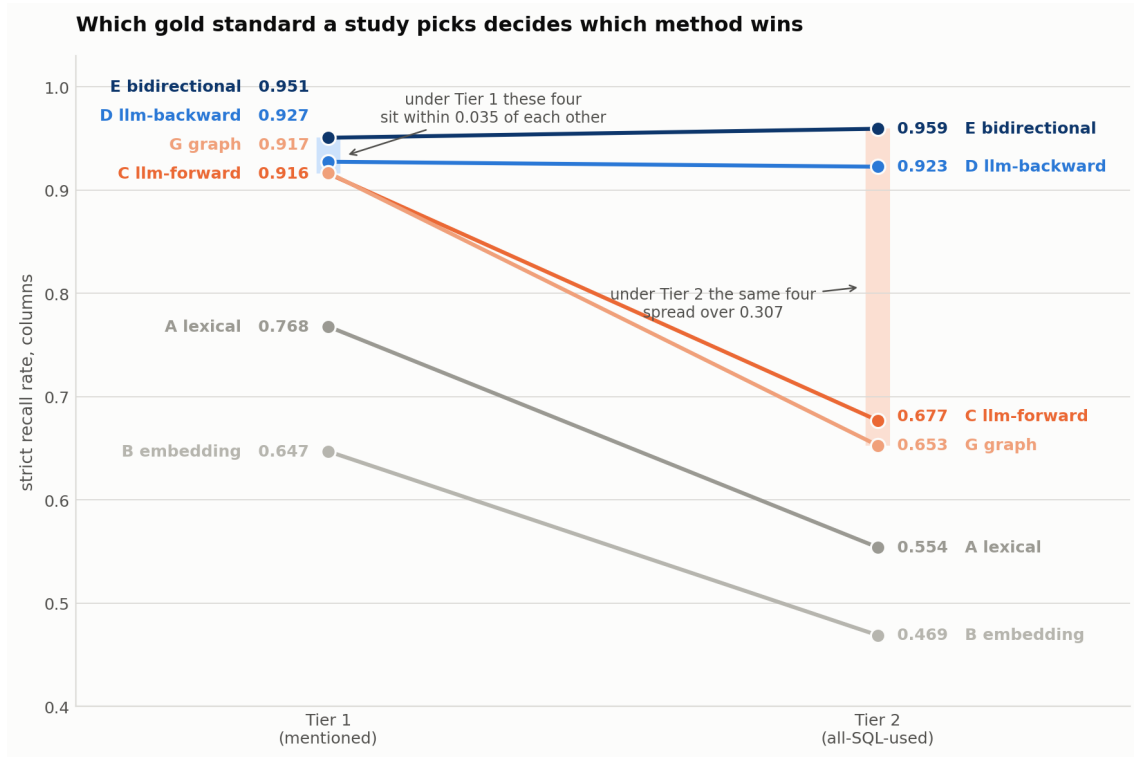


Figure 5.6: Column-level strict recall under each tier, one line per method. The four methods that sit within 0.035 of each other under Tier 1 spread over 0.307 under Tier 2. Methods D and E are flat; Methods C and G lose about a quarter of their score. The tier a study picks therefore decides which of these methods it reports as best.

5.4 Statistical Significance (Paired McNemar's Tests)

Section 4.5 defines McNemar's test, applied to eight of the fifteen possible method pairs under the same perfect-recall success criterion as SRR, computed separately per gold-standard tier and level. Table 5.4 reports the full set: eight method pairs $\times$ two tiers $\times$ two levels, 32 tests in total, each with its exact discordant-pair counts, test statistic, and p-value. The statistic is the continuity-corrected chi-square where the two discordant counts sum to 25 or more, and the smaller of the two counts, on which the exact binomial test is computed, where they sum to fewer than 25; a statistic of 0 on an exact row therefore marks a fully one-sided split rather than an absence of difference.

Figure 5.7 plots all thirty-two at once, so the eight non-significant results and the direction of every significant one can be read without going row by row.

Every non-LLM-to-LLM comparison is significant at $p < 10^{-30}$. Embedding against LLM-forward on Tier-2 columns, for example, is 28 wins to 243 losses ($\chi^2 = 169$, $p \approx 1.2 \times 10^{-38}$).

Lexical against embedding is significant but modest and direction-dependent. Embedding wins on tables (122 : 201, $p \approx 1.4 \times 10^{-5}$) and lexical wins on columns (227 : 102, $p \approx 8.1 \times 10^{-12}$), which is consistent with the complementary error profiles discussed in Section 6.1.

8 of the 32 tests are not significant ($p > 0.05$). Five involve Method G, three against Method C and two against Method E. Two more are Method C against Method E at the table level, where the two methods produce zero discordant pairs at either tier, and the last is Method C against Method D on Tier-1 columns (24 : 36, $p = 0.156$).

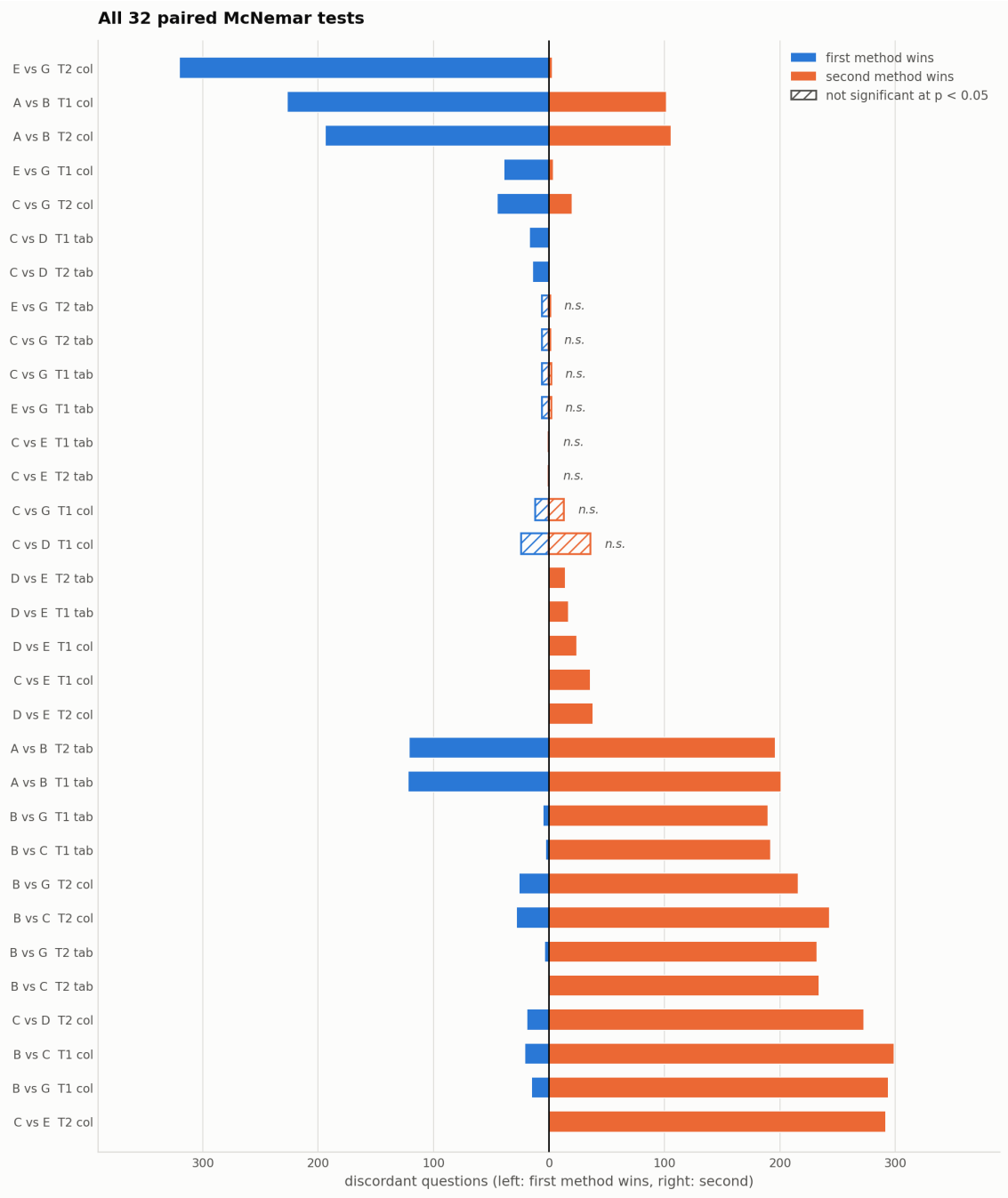


Figure 5.7: All 32 paired McNemar tests. Each bar runs from the first method's wins to the second's, so bar length is the size of the disagreement and the side is its direction. Tests that do not reach $p < 0.05$ are drawn hatched. The eight hatched bars are the eight non-significant results discussed below.

Table 5.4: Full paired McNemar results, all eight method pairs, both tiers, both levels (32 tests). Method letters follow Table 5.1. “A-only” / “B-only” are discordant-pair counts questions the first-named method got right and the second got wrong, and vice versa.

Pair (A vs B)	Tier	Level	A-only	B-only	Stats.	p-value
A vs B	T1	table	122	201	18.8	1.42e-05
		column	227	102	46.7	8.12e-12
	T2	table	121	196	17.3	3.24e-05
		column	194	106	25.2	5.09e-07
B vs C	T1	table	3	192	181.3	2.58e-41
		column	21	299	239.8	4.40e-54
	T2	table	1	234	229.0	9.66e-52
		column	28	243	169.0	1.23e-38
B vs G	T1	table	5	190	173.6	1.20e-39
		column	15	294	250.1	2.46e-56
	T2	table	4	232	218.3	2.08e-49
		column	26	216	147.6	5.78e-34
C vs D	T1	table	17	0	0.0	1.53e-05
		column	24	36	2.0	0.1556
	T2	table	14	0	0.0	0.0001
		column	19	273	219.2	1.35e-49
C vs E	T1	table	0	0	0.0	1.0000
		column	0	36	34.0	5.43e-09
	T2	table	0	0	0.0	1.0000
		column	0	292	290.0	4.96e-65
D vs E	T1	table	0	17	0.0	1.53e-05
		column	0	24	0.0	1.19e-07
	T2	table	0	14	0.0	0.0001
		column	0	38	36.0	1.95e-09
C vs G	T1	table	6	2	2.0	0.2891
		column	12	13	0.0	1.0000
	T2	table	6	1	1.0	0.1250
		column	45	20	8.9	0.0029
E vs G	T1	table	6	2	2.0	0.2891
		column	39	4	26.9	2.16e-07
	T2	table	6	1	1.0	0.1250
		column	320	3	309.2	3.34e-69

The pattern is itself informative rather than merely null. LLM-forward against graph on Tier-1 tables gives 6 : 2 discordant pairs ($p = 0.29$): Method G matches Method C’s table-level linking without an LLM ever choosing a join table, since the graph search does that instead (Section 4.2.6).

The same holds one level down. LLM-forward against graph on Tier-1 columns is the single closest test in the whole table, at 12 : 13 discordant pairs, $\chi^2 = 0.0$ and $p = 1.0000$, the maximum possible p-value. Methods C and G are statistically indistinguishable on Tier-1 columns as well as on tables. Bidirectional against graph on Tier-1 tables gives the identical 6 : 2 split for the same reason, since Method E’s table predictions equal Method C’s exactly.

Forward against bidirectional on tables has zero discordant pairs at either tier: adding Method D’s predictions to Method C’s changes nothing about which tables are found. Method E’s entire contribution over Method C is at the column level, where the union wins 292 : 0 on Tier 2 ($p \approx 5 \times 10^{-65}$). Its advantage over Method G is starker still, at 320 : 3 ($p \approx 3.3 \times 10^{-69}$), the largest discordant-pair gap in the table.

5.5 Error Analysis

Every failure in this study was categorized against a fixed set of error categories and linked to a specific question id. The population is complete rather than sampled: all 2,107 failure cases across the six methods and both gold tiers were classified manually, using the same decision criteria throughout, so the distributions reported below are exact rather than estimated.

Table 5.5 defines the eight categories and Table 5.6 gives each method’s profile against them. Three revisions to this thesis’s own prior expectations came out of that pass, and they are worth recording first, because each one changed how the numbers should be read.

First, this thesis’s own pre-analysis expectation was to record in the error code before any case was examined, treated join omission as equivalent to any Tier-2-only failure. Closer inspection showed that only 785 of the 1,100 columns present in Tier 2 but not Tier 1 (71.4%) appeared exclusively in a JOIN ON clause. The analysis was therefore revised to inspect the gold SQL directly rather than infer join omissions from a simple set difference.

Second, the draft assumed that other or corpus-related errors would account for only a small share of failures. The classified errors showed instead that such cases represented 46 to 71% of Tier-1 failures for the four strongest methods. Removing that assumption substantially changed the interpretation of the remaining errors and motivated the closer examination of benchmark quality.

Third, the draft expected over-linking to dominate the errors made by Methods A and E. The classified errors showed a different pattern: 78 to 93% of the additional tables predicted by every method were foreign-key neighbours of a gold table, indicating semantic drift rather than genuinely unrelated predictions. Once over-linking was restricted to clearly unrelated schema elements, it appeared primarily as a column-level issue for Method A.

Table 5.5 defines the eight categories used throughout the pass.

Where each method’s recall failures come from

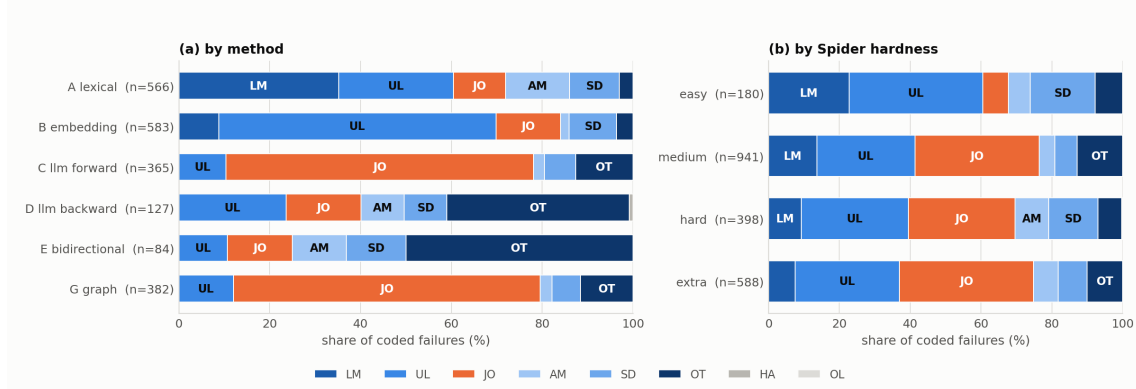


Figure 5.8: The distribution of classified recall failures. Panel (a) gives each method’s own profile. The three LLM-touching methods show no lexical mismatch at all, join omission dominates Methods C and G, and gold-standard disagreement dominates Methods D and E. Panel (b) gives the same categories by Spider hardness, showing join omission rising from 7.2% of easy-question failures to 35.1% on medium and 37.9% on extra-hard, while lexical mismatch stays roughly flat.

Figure 5.8 renders Table 5.6 as profiles rather than rows, which makes two patterns visible that the table states only implicitly. The first is that the six methods do not lie on a single quality axis: Methods A and B fail mostly by not finding things, Methods C and G by finding everything except

Table 5.5: The eight error categories. Every one of the 2,107 failure cases carries exactly one primary category from this set.

Code	Name	What it means
LM	Lexical mismatch	The question’s term shares no surface overlap with the correct element’s name, so there was nothing to match on.
UL	Under-linking	A gold element was missed that is not join-only and not covered by a more specific category.
JO	Join omission	Every missed element’s only role in the gold SQL is joining.
AM	Ambiguous phrase	The question term plausibly maps to more than one element, and the method picked a defensible but wrong one.
SD	Semantic drift	The prediction is structurally adjacent to the question, typically a foreign-key neighbour of a gold table.
OL	Over-linking	The prediction is in neither tier’s gold and has no relation to the question at all.
HA	Hallucination	The predicted element does not exist in the schema.
OT	Other / corpus error	The case fails for reasons outside the method: the gold is ambiguous, contested, or the two sources disagree.

Table 5.6: Share of each method’s classified failures by error category, as a percentage of that method’s own n. Lexical mismatch disappears entirely once a model is in the loop, and join omission dominates Methods C and G. Categories are defined in Table 5.5.

Method	LM	UL	JO	AM	SD	OT	HA	OL	n
A lexical	35.2	25.3	11.5	14.1	11.0	3.0	–	–	566
B embedding	8.7	61.1	14.2	1.9	10.5	3.6	–	–	583
C llm forward	–	10.4	67.7	2.5	6.8	12.6	–	–	365
D llm backward	–	23.6	16.5	9.4	9.4	40.2	0.8	–	127
E bidirectional	–	10.7	14.3	11.9	13.1	50.0	–	–	84
G graph	–	12.0	67.5	2.6	6.3	11.5	–	–	382

the join keys, and Methods D and E by being scored against gold elements their predictions had no way to recover.

The second is that hardness sharpens these profiles without reordering them (panel b). Join omission grows with question complexity because multi-table SQL is where joins live, while lexical mismatch stays roughly flat.

Three findings follow from Table 5.6, each answering part of the question the aggregate metrics leave open. First, Methods C and G are join-omission machines: 67.7% and 67.5% of their failures are join omissions, otherwise-perfect predictions that dropped the connecting keys.

The dominant sub-pattern, tagged one-sided-fk and the single most common tag in the study at 742 cases, is that the method predicts the child-side foreign-key column and omits the parent key it points at. It is a systematic blind spot rather than scattered noise, and it is exactly what Tier-2 evaluation (Section 3.2.2) was designed to expose.

A representative case: on `concert_singer` question 22, Method C’s forward prediction is missing only `stadium.stadium_id`, every other required element being correct, and the recorded rationale states plainly that the sole remaining miss is a foreign key with no other role in the query. The same one-sided pattern recurs on question 31, where Method C additionally predicts an unneeded extra table (`singer_in_concert`) alongside the same missing join key, over-inclusion and join omission occurring in the same prediction.

Second, the embedding ceiling is a budget problem, not a semantics problem. 61.1% of Method B’s failures are under-linking, with a cutoff tag on 484 cases, and of its Tier-1 column misses, 375 cleared the similarity threshold and lost only to the `column_top_k = 3` cap. Only 92 fell below the threshold at all.

Questions 2 and 3, both against `concert_singer`, show the shape directly. Method B misses `singer.name` and `singer.country` respectively, while still predicting `singer.singer_id` and the `singer_in_concert` table: the encoder ranked the right table correctly but let the specific gold column fall just outside its `column_top_k = 3` cutoff. The encoder usually knows the right answer and the operating point throws it away, so raising `k` is a larger lever than a better encoder would be.

Third, for Methods D and E the gold standard itself is the largest single error source. Other or corpus error accounts for 40.2% of Method D’s failures and 50.0% of Method E’s, cases where Tier-1 gold names an element the gold SQL never references.

Questions 6 and 7, also against `concert_singer`, are representative: Method E is missing `singer.name` in both, classified as corpus error because that element is absent from the gold SQL entirely. Question 6 is the same case examined directly in Section 3.2’s worked example, where Taniguchi et al.’s annotation marks `singer.Name` for a question whose SQL actually selects `song_name`. Section 3.2’s illustration of tier divergence and this section’s classified failure are the same underlying event, seen from two different angles.

Chapter 6

Discussion

Chapter 5 established what each method does. This chapter asks what that means, taking the four key questions in turn: whether string matching or embeddings makes the better baseline (Section 6.1), whether the LLM-based methods justify their cost (Section 6.2), whether the graph method closes the gap it was built to close (Section 6.3), and what each family’s characteristic failure says about where the remaining headroom is (Section 6.4).

6.1 String Matching versus Embeddings

Neither baseline dominates the other, and the difference is one of failure mode rather than overall quality. Embeddings win on tables (Tier-1 SRR 0.791 vs. 0.715, $p \approx 1.4 \times 10^{-5}$; Section 5.4) because table names are short, semantically loaded strings an encoder handles well. Lexical matching wins on columns (0.768 vs. 0.647, $p \approx 8.1 \times 10^{-12}$) but only by brute force: Method A predicts 4.49 columns per question at precision 0.403, effectively buying recall with false positives rather than winning on semantic quality.

Their error profiles, reported in Table 5.6, are near-complements. Method A’s failures are 35.2% lexical mismatch: it has no way to connect a question’s “gender” to a column named Sex. Method B’s are 61.1% under-linking, and Section 5.5’s second finding shows this is almost entirely top-k truncation of elements the encoder already scored correctly, rather than a semantic failure.

A hybrid taking Method B’s semantic matching and Method A’s generosity at the retrieval stage is the obvious unexplored point between them, and is named explicitly in this thesis’s future work (Section 7.3).

6.2 Do LLM Methods Win, and at What Cost?

Yes, decisively, and cheaply. Every LLM-based method beats every non-LLM baseline on every headline metric at $p < 10^{-30}$ (Section 5.4). The gap is largest exactly where it matters most: on extra-hard questions under Tier 2, Method E holds 0.892 column SRR against Method B’s 0.108 (Table 5.2).

The more interesting result is which LLM method wins, and it is not the obvious one. Direct prompting (Method C) is the weakest of the three LLM-based methods on Tier 2. Backward parsing (Method D), which asks the model for SQL and then reads the links off it, wins Tier-2 columns outright at F1 = 0.939 against Method C’s 0.815, for roughly half the cost (Section 4.3.3).

The reason is structural: join keys are a property of a real query rather than something the model has to be told to remember.

The union (Method E) buys the best recall in this study for no extra API calls, at a precision cost of roughly 8 points on Tier-1 columns.

Cost turns out not to be a meaningful constraint at this scale. The complete LLM budget for this thesis was \$11.89 across 6,767 calls (Chapter 4), averaging roughly \$0.006 per question for the single most expensive method. Prompt caching was not enabled for these runs, so this figure is an unoptimized upper bound rather than a tuned minimum.

At this price point, the relevant budget for a deployment decision is latency and vendor dependency rather than dollars. That reframes the “death of schema linking” debate this thesis’s Sections 1.1 and 2.2 situate it against. Maamari et al. (2024) argue the cost-benefit case for omitting schema linking turns on whether an imperfect linker’s precision loss outweighs its safety value for a given model and schema size.

This thesis’s own cost data suggests that for a small model on Spider-scale schemas, the dollar cost of running a linker at all is not the variable doing the deciding. Which method is used, and in which direction it reasons, matters far more than whether one is run.

6.3 Does the Graph Method Close the Gap?

For tables, completely. Method G matches Method C on Tier-1 tables with no significant difference (6 : 2 discordant pairs, $p = 0.29$; Section 5.4), and does so with one deterministic model call per question instead of three sampled ones.

Its endpoint model produced zero hallucinations across all 1,034 development queries, and it is the graph’s foreign-key path search, not the model, that chooses the join tables, inserting an extra table on only 33 queries and usually correctly (Section 5.2). On Tier-1 columns, Method G posts the best precision (0.698) and F1 (0.757) of any method in this study, for \$1.93 in total (Chapter 5).

For columns under Tier 2, no. Method G’s columns still come from the same LLM endpoint call that names its core tables, and that call omits join keys exactly as Method C’s forward prediction does. 67.5% of Method G’s failures are join omissions (Table 5.6), and its Tier-2 column SRR of 0.653 is statistically indistinguishable from the lexical baseline on extra-hard questions specifically (0.313 against Method A’s 0.313; Table 5.2).

The graph structure is used to find the join tables but never to emit the join columns, even though the foreign-key edges Method G already walks contain exactly the column pairs its predictions are missing. That claim can be measured. Of the 657 Tier-2 columns Method G fails to predict, 585 (89.0%) lie on a foreign-key edge between two tables its own path search already walked, and 319 of the 359 affected questions (88.9%) contain at least one such column. The information really is sitting in a structure the method builds anyway.

Emitting them is a cheap change to make. Adding every foreign-key column on a walked edge, which needs no model call and no information the method does not already hold, raises Method G’s Tier-2 column strict recall from 0.653 to 0.947 and its F1 from 0.808 to 0.885, at a precision cost of one thousandth. That would place it second of the six methods rather than fourth, behind only Method E. Under Tier 1 the same change costs precision (0.698 to 0.610), since Tier 1 does not record join keys at all, which is a further instance of the tier argument of Section 5.3.

Closing that loop is therefore the clearest single improvement available in this study, and it is named first among this thesis's near-term future work (Section 7.3), precisely because the information is already sitting in a data structure the method builds anyway.

6.4 Dominant Error Categories

Table 5.6 and Section 5.5's three findings show that each method family fails in its own characteristic way, and none of the six failure profiles is well described as simply "less accurate".

Method A's signature is lexical mismatch (35.2%) then under-linking (25.3%), with a distinctive wrong-table-copy pattern, a correctly named column predicted on the wrong table, on 300 of 1,034 queries. Method B's is under-linking (61.1%), overwhelmingly rank truncation rather than semantic failure. Methods C and G share join omission (67.7% and 67.5%) as their dominant category, with the one-sided foreign-key pattern as its most common shape. Methods D and E share other or corpus error (40.2% and 50.0%) as theirs.

Question hardness sharpens these profiles rather than changing them. Join omissions cluster heavily in the medium and extra-hard strata (330 of 941 and 223 of 588 coded cases there, against 13 of 180 on easy questions), which is where multi-table SQL actually lives; lexical mismatch, by contrast, is roughly flat across hardness, since a vocabulary gap between question and schema does not care how syntactically complex the resulting query is.

Chapter 7

Conclusion and Future Work

This thesis set out to compare six schema linking method families, lexical, embedding-based, three LLM-based reasoning directions, and graph-based, under one shared protocol, one dataset, and a common set of metrics, addressing the absence of exactly this kind of controlled comparison.

This chapter summarizes what was found against the key questions posed, restates this thesis's contributions in light of what is complete and what remains outstanding, and sets out the work that follows from it.

7.1 Summary of Findings

Schema linking on Spider is not one task with one number. Measured against what the question mentions, four of the six methods are effectively at ceiling: Methods C, D, E and G sit between 0.916 and 0.951 column strict recall, a spread of 0.035.

Measured against what the SQL actually needs, the same four methods spread across thirty points of strict recall, and the ordering is determined almost entirely by one structural property: whether the method's output passed through a real SQL query on its way out.

The strongest practical result is that backward extraction beats direct prompting. Asking a small, cheap model for a throwaway SQL query and parsing the links out of it outperformed asking the same model for the links directly, on the metric that governs downstream executability, at half the cost.

The strongest methodological result is that a benchmark's residual error can come to be dominated by its own annotation long before anyone notices. Here that happens at around 0.92 strict recall, a level several published systems already exceed.

7.2 Contributions

This thesis claimed four contributions at the outset (Section 1.3). This section restates each and assesses it against what Chapters 3 through 6 actually delivered.

- A hybrid, two-tier gold-standard schema-linking annotation for Spider. Delivered in full (Chapter 3): Tier 1 draws on Taniguchi et al. (2021) complete annotated development
-

set; Tier 2 is extracted programmatically via sqlglot across both training and development splits. Both tiers were used throughout Chapter 5’s evaluation, and Section 5.3’s tier-asymmetry finding is a direct product of having built both.

- A shared linker protocol that all six evaluated methods implement. Delivered in full (Section 4.3.1): all six methods return predictions through the same interface, scored by the same evaluator, which is what made Chapter 5’s like-for-like comparison possible rather than six separate, potentially inconsistent evaluation scripts.
- An evaluation framework combining five metrics with McNemar’s test. Delivered in full: precision, recall, F1, SRR and F6 are computed and reported for every method and gold-standard tier (Chapter 5), and the McNemar significance tests are run and reported for all eight method pairs across both tiers and both levels, 32 tests in total (Section 5.4). The one piece of this contribution an earlier draft had marked as outstanding is now complete.
- A controlled empirical comparison of six schema linking method families under an explicit cost budget. Delivered (Chapter 5; Section 6.2): all six families were compared under identical conditions, for a total measured cost of \$11.89, and that cost data is itself one of this thesis’s findings rather than merely a constraint that was satisfied.

7.3 Future Work

Close the loop in the graph method. Method G already computes a foreign-key path between the tables a question needs, and every edge on that path is a pair of columns. It does not emit them: its columns come from the same model call that names its core tables, and that call is instructed to withhold join keys. Of the 657 Tier-2 columns Method G fails to predict, 585 (89.0%) lie on an edge between two tables its own search already walked.

Emitting them requires no model call and no information the method does not already hold, and would raise its Tier-2 column strict recall from 0.653 to 0.947 at a precision cost of one thousandth (Section 6.3). That is the largest single improvement this study identifies, and the cheapest. It is listed first for both reasons.

Combine the two training-free baselines. Section 6.1 showed that Methods A and B fail in near-complementary ways: Method A’s failures are dominated by lexical mismatch, which an encoder handles, while Method B’s are dominated by rank truncation of elements it had already scored correctly. A hybrid that keeps Method B’s semantic scoring but adopts Method A’s generosity at the retrieval stage, by raising or removing the top-k cap, is the obvious unexplored point between them. Section 4.4.1’s tuning grid suggests the cap rather than the threshold is the binding constraint, so this is a cheap experiment.

Run the deferred open-model ablation, and evaluate on a messier benchmark. Both were excluded from this thesis’s scope, and both bear on how far the findings generalize. The open-model ablation would separate the broad claim that LLMs solve this task from the narrower possibility that these results are specific to Claude Haiku 4.5. Testing on a less curated benchmark such as BIRD would show whether Spider’s relatively clean, English-native schemas are part of why Method A performs as well as it does at the column level.

Scale to real-world databases. Spider’s schemas average 5.28 tables and 27.13 columns (Section 3.1), small enough that every method evaluated here can be shown the entire schema in one prompt. Production databases routinely run to hundreds of tables with abbreviated or legacy naming, undeclared or partial foreign keys, and no curated gold SQL to grade against. That setting changes the problem rather than merely scaling it: once the full schema no longer fits in

context, retrieval becomes part of the linker rather than a step it can skip, which is the regime LinkAlign (Wang, Liu, & Yang, 2025) and AutoLink (Wang et al., 2025b) were built for. Whether the ranking measured here survives that shift, and in particular whether Method G’s graph search gains or loses ground when the foreign-key graph is large and incompletely declared, is the most consequential open question this thesis leaves.

Measure downstream execution accuracy. Deliberately cut from this thesis’s scope, this is the natural next study. It would test whether the thirty-point Tier-2 spread measured here at the linking stage translates into generated-SQL correctness. That assumption underpins the whole schema-linking literature reviewed in Chapter 2, and this thesis measures its input rather than its output.

Taken together, the findings of this thesis provide clear and statistically supported answers to its research questions. The work was motivated by the gap between schema linking’s well-established importance to Text-to-SQL performance (Chapter 1) and the inconsistent, difficult-to-compare ways it has been evaluated in the literature (Chapter 2).

By evaluating six methods under a common protocol, against two explicit definitions of what a correct link is, this study closes part of that gap. More broadly, the results show the value of treating schema linking as an evaluation problem in its own right, rather than only as an intermediate component of an end-to-end Text-to-SQL system.

References

- R. Cai, J. Yuan, B. Xu, and Z. Hao. SADGA: Structure-Aware Dual Graph Aggregation Network for Text-to-SQL. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, pages 7664–7676, 2021.
- R. Cao, L. Chen, Z. Chen, Y. Zhao, S. Zhu, and K. Yu. LGESQL: Line Graph Enhanced Text-to-SQL Model with Mixed Local and Non-Local Relations. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL-IJCNLP)*, pages 2541–2555, 2021.
- Z. Cao, Y. Zheng, Z. Fan, X. Zhang, W. Chen, and X. Bai. RSL-SQL: Robust Schema Linking in Text-to-SQL Generation. *arXiv preprint arXiv:2411.00073*, 2024.
- Z. Chen, L. Chen, Y. Zhao, R. Cao, Z. Xu, S. Zhu, and K. Yu. ShadowGNN: Graph Projection Neural Network for Text-to-SQL Parser. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 5567–5577, 2021.
- X. Dong, C. Zhang, Y. Ge, Y. Mao, Y. Gao, L. Chen, J. Lin, and D. Lou. C3: Zero-Shot Text-to-SQL with ChatGPT. *arXiv preprint arXiv:2307.07306*, 2023.
- M. Glass, M. Eyceoz, S. Subramanian, G. Rossiello, L. Vu, and A. Gliozzo. Extractive Schema Linking for Text-to-SQL. *arXiv preprint arXiv:2501.17174*, 2025.
- J. Guo, Z. Zhan, Y. Gao, Y. Xiao, J.-G. Lou, T. Liu, and D. Zhang. Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4524–4535, 2019.
- G. Katsogiannis-Meimarakis, K. Mirylenka, P. Scotton, F. Fusco, and A. Labbi. In-Depth Analysis of LLM-Based Schema Linking. In *Proceedings of the 29th International Conference on Extending Database Technology (EDBT 2026)*, 2026.
- W. Lei, W. Wang, Z. Ma, T. Gan, W. Lu, M.-Y. Kan, and T.-S. Chua. Re-examining the Role of Schema Linking in Text-to-SQL. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6943–6954, 2020.
- H. Li, J. Zhang, C. Li, and H. Chen. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence*, pages 13067–13075, 2023.
- X. Liu, S. Shen, B. Li, P. Ma, R. Jiang, Y. Zhang, J. Fan, G. Li, N. Tang, and Y. Luo. A Survey of Text-to-SQL in the Era of LLMs: Where are we, and where are we going. *arXiv preprint arXiv:2408.05109*, 2024.
- K. Maamari, F. Abubaker, D. Jaroslawicz, and A. Mhedhbi. The Death of Schema Linking? Text-to-SQL in the Age of Well-Reasoned Language Models. *arXiv preprint arXiv:2408.07702*, 2024.
- Q. McNemar. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2):153–157, 1947.
-

- B. Qin, B. Hui, L. Wang, M. Yang, J. Li, B. Li, R. Geng, R. Cao, J. Sun, L. Si, F. Huang, and Y. Li. A Survey on Text-to-SQL Parsing: Concepts, Methods, and Future Directions. *arXiv preprint arXiv:2208.13629*, 2022.
- A. Safdarian, M. Mohammadi, E. Jahanbakhsh, M. S. Naderi, and H. Faili. SchemaGraphSQL: Efficient Schema Linking with Pathfinding Graph Algorithms for Text-to-SQL on Large-Scale Databases. *arXiv preprint arXiv:2505.18363*, 2025.
- Y. Taniguchi, H. Nakayama, K. Takahiro, and J. Suzuki. An Investigation Between Schema Linking and Text-to-SQL Performance. *arXiv preprint arXiv:2102.01847*, 2021.
- B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7567–7578, 2020.
- Y. Wang, P. Liu, and X. Yang. LinkAlign: Scalable Schema Linking for Real-World Large-Scale Multi-Database Text-to-SQL. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 977–991, 2025a.
- Z. Wang, Y. Zheng, Z. Cao, X. Zhang, Z. Wei, P. Fu, Z. Luo, W. Chen, and X. Bai. AutoLink: Autonomous Schema Exploration and Expansion for Scalable Schema Linking in Text-to-SQL at Scale. *arXiv preprint arXiv:2511.17190*, 2025b.
- S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff. C-Pack: Packaged Resources to Advance General Chinese Embedding. *arXiv preprint arXiv:2309.07597*, 2023.
- S. Yang, Q. Su, Z. Li, Z. Li, H. Mao, C. Liu, and R. Zhao. SQL-to-Schema Enhances Schema Linking in Text-to-SQL. In *Proceedings of Database and Expert Systems Applications (DEXA 2024)*, pages 139–145. Springer, 2024.
- T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921, 2018.
-

Appendix A

Gold-Link JSON Schema

Both gold-standard tiers (Chapter 3) are materialized as JSON files keyed by Spider question ID, one entry per question, in the canonical shape shown below, question id "0" ("How many singers do we have?", database `concert_singer`):

```
{
  "db_id": "concert_singer",
  "tables": [
    "singer"
  ],
  "columns": []
}
```

Listing A.1: Canonical gold-link record, question id 0.

“columns” holds [table, column] pairs rather than bare column names, so a column reference remains unambiguous even when the same column name is reused across tables in a schema.

Appendix B

Prompt Templates for LLM-Based Methods

The system prompts below are the exact, locked text used to produce this thesis’s reported results, reproduced verbatim. Each method also uses two fixed few-shot examples (one single-table pattern, one multi-table join pattern), drawn from Spider train and excluded from every iteration and evaluation set so no method is ever tested on an example baked into its own prompt.

B.1 Method C - forward_v1

```
You extract the tables and columns from a database schema that are relevant to answering a
natural-language question.

Rules:
1. Use ONLY tables and columns that appear in the provided schema.
2. Return every table you need. Return every column you need.
3. Do NOT return columns whose ONLY purpose is joining tables together - return only
   columns that would appear in SELECT, WHERE, GROUP BY, HAVING, or ORDER BY of the target
   SQL.
4. Output ONLY a JSON object with keys "tables" (list of strings) and "columns" (list of [
   table_name, column_name] pairs). No prose, no explanation, no markdown fences.
```

Listing B.1: System prompt forward_v1 (Method C).

A revised forward_v2, rewriting rule 3 to state the inclusion criterion positively (“a table or column must correspond to something the target SQL actually references...”) rather than only negatively, was drafted and re-run on the same 20-example train iteration set. It did not reproducibly outperform forward_v1, across repeated iteration runs the two versions’ aggregate F1 gap (0.005–0.008) flipped sign three times, smaller than this 20-example set’s own noise floor and forward_v1 was retained as the frozen prompt for the reported dev run.

B.2 Method G - graph_endpoint_v1

```
You identify the 1 to 3 core tables and the specific columns needed from a database schema
to answer a natural-language question.

Rules:
```

1. Return between 1 and 3 core tables. Prefer fewer.
2. Do NOT include tables that only exist to join others - those will be added automatically by a graph algorithm.
3. For columns, return only those that would appear in SELECT, WHERE, GROUP BY, HAVING, or ORDER BY of the target SQL. Not join columns.
4. Every table and column must be from the provided schema.
5. Output ONLY a JSON object with keys "core_tables" (list of 1-3 strings) and "columns" (list of [table_name, column_name] pairs).

Listing B.2: System prompt graph_endpoint_v1 (Method G).

A revised graph_endpoint_v2, adding an explicit join-only test to rule 2, was drafted and tested the same way; it likewise produced no reproducible improvement on the 20-example train iteration set, and graph_endpoint_v1 was retained.

B.3 Method D - backward_v1

You are a SQL expert. Given a database scheme and a natural-language question, generate a single SQL query that would answer the questions.

Rules:

1. Use ONLY tables and columns that appear in the provided schema.
2. Return only the SQL query. No explanation, no markdown fences, no prose
3. Do not add comments inside the SQL.
4. If the question is ambiguous, choose the most literal interpretation.

Listing B.3: System prompt backward_v1 (Method D).

Method D's prompt instructs Claude Haiku 4.5 to generate a single SQL query answering the question against the full rendered schema, with no self-consistency and no output-format rules beyond producing valid SQL, the schema-linking prediction is then read off that generated query by the same sqlglot-based extractor used for Tier 2 gold (Section 3.2.2), rather than by any rule in the prompt itself.

Appendix C

Further Worked Schema-Linking Examples

Figure 3.5 shows the core divergence between the two gold tiers on one question, and Figure 3.11 the reverse case. The three figures below complete the set, in the same form: each adds one construct the sqlglot walk of Section 3.2.2 must handle. All five are generated from the released gold files rather than transcribed, so a figure cannot claim a link the gold standard no longer holds.

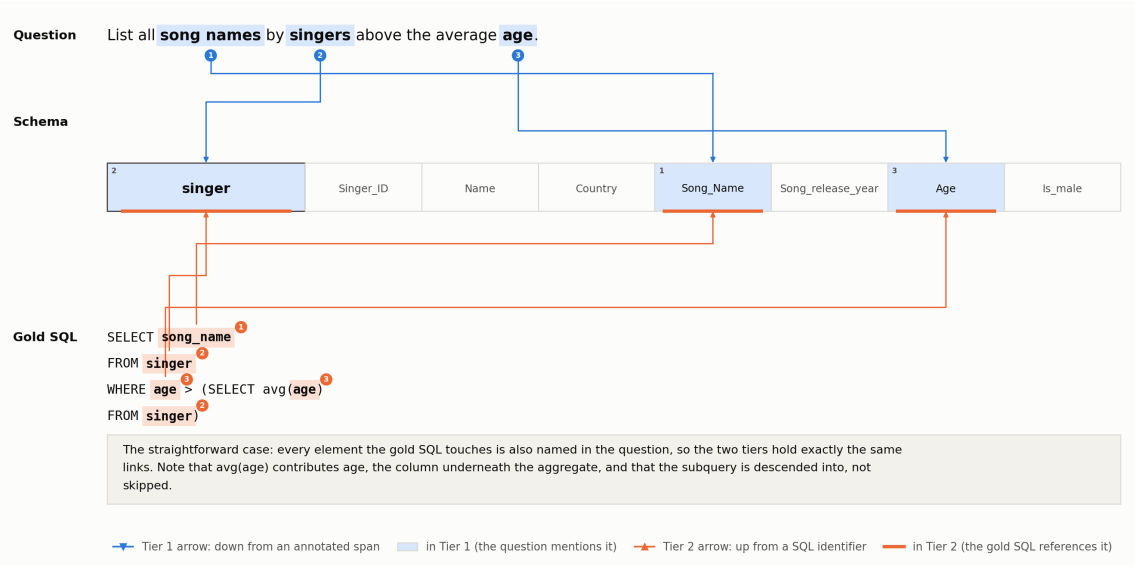


Figure C.1: Development question 12 (concert_singer): the straightforward case, in which every element the gold SQL touches is also named in the question, so the two tiers hold exactly the same links. Note that avg(age) contributes age, the column underneath the aggregate, and that the subquery is descended into rather than skipped.

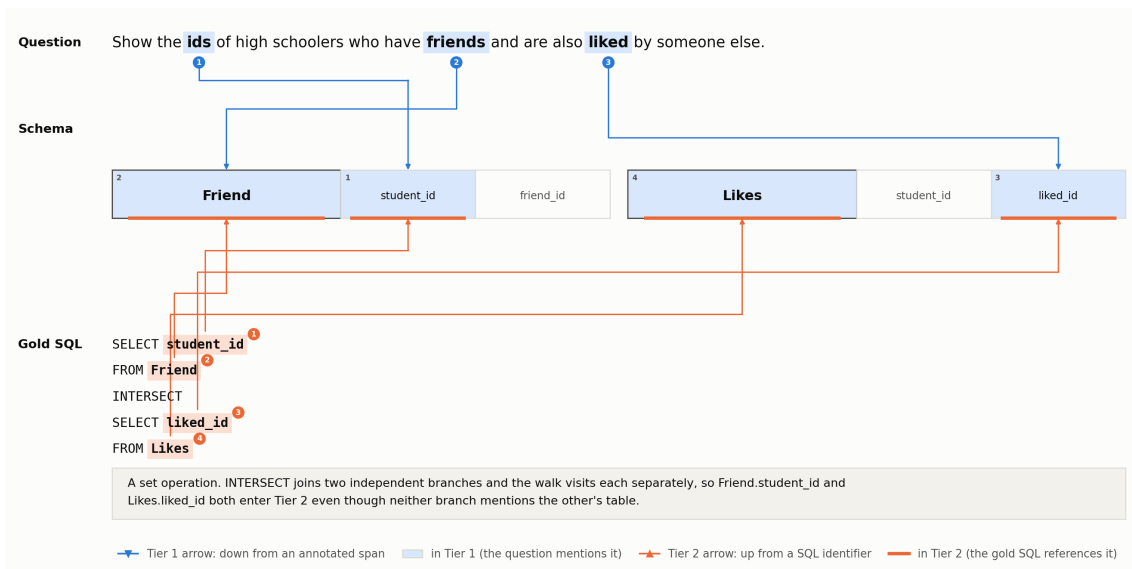


Figure C.2: Development question 898 (network_1): a set operation, with each branch of the query visited separately by the walk.

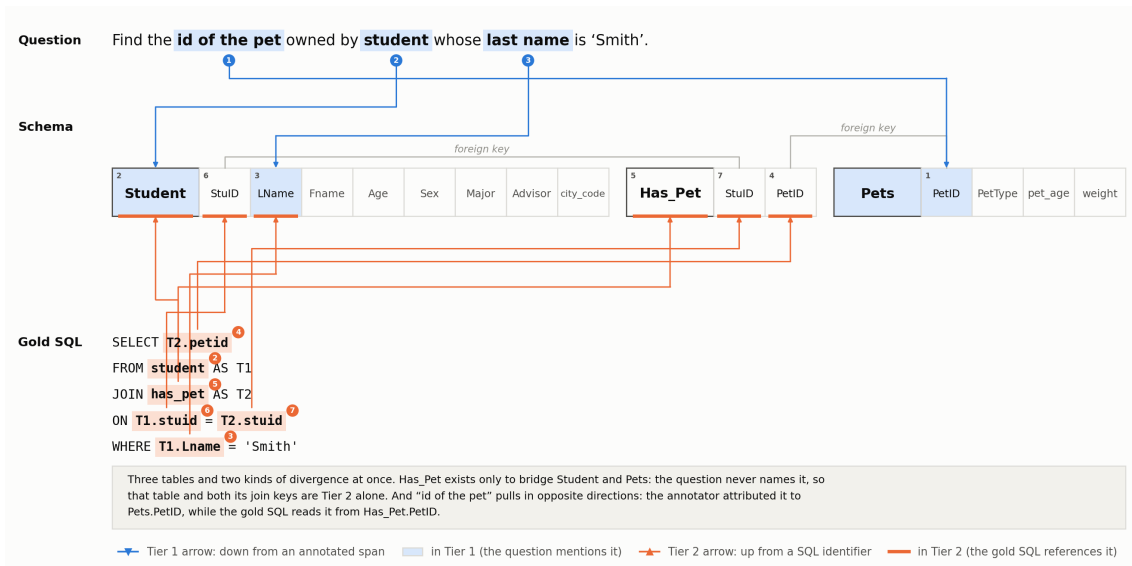


Figure C.3: Development question 77 (pets_1): a three-table join whose bridge table the question never names, the case that produces the 88 tables Tier 2 holds and Tier 1 lacks (Section 3.3).

Appendix D

Implementation Reference

Section 4.3 describes the three components every method shares: the linker protocol, the SQL-parsing utility, and the LLM client. The two tables below record where that code lives and which third-party libraries it depends on. They are reference material rather than argument, which is why they sit here rather than in the chapter itself.

Table D.1: The codebase, by role. Roughly 16,100 lines of Python under `src/schema_linking/`, exercised by 740 tests under `tests/`.

Role	Module	What it does
Contract	<code>base.py</code>	The Linker protocol, the Prediction record, and the score records
Data	<code>data_loader.py</code> , <code>schema_parser.py</code>	Load Spider’s JSON into typed records; build Schema objects, excluding the wildcard column at source (Section 3.2.2)
Gold	<code>gold_link_extractor.py</code> , <code>taniguchi_loader.py</code>	Tier 2 by sqlglot walk; Tier 1 by the Taniguchi join and its id-offset fallback (Section 3.2.1)
Method A	<code>lexical_linker.py</code>	Substring / token / fuzzy matching
Method B	<code>embedding_linker.py</code> , <code>utils/embeddings.py</code>	Encoder, rendering rules, per-database vector cache, top-k + threshold
Methods C, D, E	<code>llm_linker.py</code> , <code>utils/prompts.py</code> , <code>utils/fewshot.py</code>	Prompt registry, self-consistency voting, SQL-backward extraction, the CUD union
Method G	<code>graph_linker.py</code> , <code>utils/graph.py</code>	Endpoint resolution, shortest path, greedy Steiner approximation
Shared SQL	<code>utils/sql_parsing.py</code>	One sqlglot walk in strict and permissive modes (Section 4.3.2)
Evaluation	<code>evaluator.py</code> , <code>utils/statistical.py</code>	P/R/F1/F6/SRR/HR, macro and micro; McNemar’s test
Tuning	<code>utils/tuning.py</code> , <code>utils/difficulty.py</code>	The Method A sweep and the Method B grid; Spider hardness labels
Cost	<code>utils/llm_client.py</code>	Retries, the JSONL call ledger, the per-model pricing table
Error analysis	<code>error_analysis.py</code> , <code>auto_coder.py</code> , <code>gold_audit.py</code>	The codebook cascade of Section 5.5 and the Tier-1 gold audit
Config	<code>utils/config.py</code> , <code>config.yaml</code>	Every path, threshold and pinned model revision in one file

Table D.2: Third-party libraries, and why each was chosen.

Library	Used for	Why this one
sqlglot	Every SQL parse - Tier 2 gold and Method D alike	A real dialect-aware parser with an inspectable AST; regex or sqlparse cannot resolve aliases, subqueries, or set operations (Table 3.1).
rapidfuzz	Method A's fuzzy strategy	Reference implementation for BGE checkpoints, with revision pinning.
sentence-transformers + torch	Method B's encoder	
networkx	Method G's foreign-key graph	Standard shortest-path primitives; the graph is built once per schema.
anthropic + tenacity	Methods C, D, and G	Official API client; tenacity supplies the typed retry policy described in Section 4.3.3.
pandas, numpy	Every results table and sweep	—
scipy, statsmodels		Exact binomial test on the discordant cells.
scikit-learn	Cohen's κ for the error-analysis agreement pass	—
matplotlib	Every figure	—
pytest	Creating Tests cases	Every metric, extractor, and parsing rule in Table 3.1 is pinned by a test.